

Capitolo 1: Concetti e Principi Fondamentali della Sicurezza Informatica

NECKER

17 Settembre 2025

Indice

1	Introduzione	9
2	Obiettivi Fondamentali della Sicurezza Informatica	9
2.1	Proprietà fondamentali (Security Properties)	9
2.2	Terminologia	10
3	Politiche di Sicurezza e Attacchi	10
3.1	Security Policy	10
3.2	Attacchi e Minacce	10
3.3	Esempio non tecnico	11
4	Rischio e Valutazione del Rischio	11
4.1	Definizioni	11
4.2	Equazioni del rischio	11
4.3	Expected Loss Modeling	11
4.4	Domande chiave per la valutazione del rischio	12
4.5	Cost-Benefit Analysis	12
4.6	Valutazione qualitativa	12
4.7	Gestione del rischio	12
5	Modellazione dell'Avversario e Analisi della Sicurezza	12
5.1	Attributi di un adversary	12
5.2	Classificazione degli avversari (Named Groups)	13
5.3	Schemi di classificazione	13
5.4	Security Evaluation e Penetration Testing	13
5.5	Security Analysis	14
5.6	Security Models	14

6	Threat Modeling: Diagrams, Trees, Lists and STRIDE	14
6.1	Definizione generale	14
6.2	Approcci principali	14
6.3	Diagram-driven Threat Modeling	15
6.4	Attack Trees	15
6.5	Checklists e STRIDE	16
7	Model-Reality Gaps and Real-World Outcomes	16
7.1	Difficoltà del Threat Modeling	16
7.2	Esempi pratici	16
7.3	Iterative Process: Evolving Threat Models	17
7.4	Security Policy e Real-World Outcomes	17
7.5	Security Analysis: Key Questions	17
7.6	Testing e Assurance	17
8	1.7 Design principles for computer security	18
8.1	Prefazione	18
8.2	Principi (P1–P20)	18
8.3	Principi di alto livello e massima	19
9	Capitolo 3: User Authentication — Passwords, Biometrics and Alternatives	20
9.1	3.1 Password authentication: concetti chiave	20
9.2	Pre-computed dictionary attack (attacco a tavole pre-calcolate)	20
9.3	Targeted vs Trawling scope	20
9.4	Approcci per sconfiggere l'autenticazione basata su password .	20
9.5	Password composition, vantaggi e svantaggi	21
10	3.2 Password-guessing strategies and defenses	21
10.1	Online password guessing & rate-limiting	21
10.2	Offline password guessing	21
10.3	Iterated hashing (password stretching)	22
10.4	Password salting	22
10.5	Pepper (secret salt)	22
10.6	Specialized password-hashing functions (KDFs)	22
10.7	System-assigned passwords (brute-force) e probabilità di successo	22
10.8	User-chosen passwords e distribuzioni sbilanciate	23
10.9	Login passwords vs passkeys	23
10.10	Tabella riassuntiva (misure vs attacchi)	23

11 3.3 Account recovery and secret questions	23
11.1 Meccanismi comuni di recovery	23
11.2 Problemi di sicurezza e usabilità	24
11.3 Esempio: password reset attack (interleaving)	24
12 3.4 One-time password generators and hardware tokens	24
12.1 Motivazione	24
12.2 Modalità comuni di OTP	24
12.3 Vettori d'attacco specifici: SIM swap	25
12.4 Passcode Generators	25
12.5 Hardware Tokens	25
12.6 User Authentication Categories	25
12.7 Multi-Factor Authentication (MFA)	26
13 Biometric Authentication	26
13.1 Motivation	26
13.2 Types of Biometrics	26
13.3 Enrollment and Verification Process	26
13.4 Error Types and Tradeoffs	27
13.5 Biometric Evaluation Criteria	27
13.6 Circumvention and Attacks	27
13.7 Authentication vs Identification	28
14 The Reference Monitor, Access Matrix, and Security Kernel	29
14.1 Concetto di Reference Monitor	29
14.2 Access Matrix Model	29
14.3 Implementazione del Reference Monitor	30
14.4 Requisiti del Reference Validation Mechanism	30
14.5 Security Kernel	30
14.6 Dipendenze del Reference Monitor	30
14.7 Implementazione dell'Access Matrix	31
14.8 Capability Lists (C-Lists)	31
14.9 Access Control Lists (ACLs)	31
14.10 Capability-Based vs ID-Based Protection	32
14.11 Implementazione delle Capability Systems	32
14.12 Access Control e Audit Trails	32
14.13 Access Control tramite Encryption e Key Release	33
15 5.4 Implementation Issues	34
15.1 5.4.1 Aspetti generali	34
15.2 5.4.2 Subjects: Users	34

15.3	5.4.3 Superuser	34
15.4	5.4.4 Groups	35
15.5	5.4.5 Assegnazione UID ai processi	35
15.6	5.4.6 Creazione dei processi	35
15.7	5.4.7 System Call <code>fork()</code>	36
15.8	5.4.8 Objects in UNIX	37
15.9	5.4.9 Objects in Windows	37
16	5.5 Privilege Escalation	37
16.1	5.5.1 Definizione	37
16.2	5.5.2 Password Dilemma	38
16.3	5.5.3 Set-UID Concept	38
16.4	5.5.4 Implementazione Set-UID	38
16.5	5.5.5 Come abilitare Set-UID	39
16.6	5.5.6 Funzionamento con <code>fork()</code> e <code>exec()</code>	39
17	5.6 Object Permissions e File-Based Access Control	40
17.1	5.6.3 ACL Alternatives	40
17.2	5.6.4 File Owner e Group	40
17.3	5.6.5 Superuser vs Root	40
17.4	5.6.6 User-Group-Others (ugo) Model	40
17.5	5.6.7 Meta-data e File Permissions	41
17.6	5.6.8 Uso dei Protection Bits	41
17.7	5.6.9 Permission Display Notation	41
17.8	5.6.10 Valori iniziali dei Protection Bits	42
18	5.7 Setuid Bit e Effective UserID (eUID)	42
18.1	5.7.3 Setgid Bit	42
18.2	5.7.4 Esempio Setuid: <code>passwd</code>	42
18.3	5.7.5 Inheriting UserIDs	42
18.4	5.7.6 Esempio UserIDs e <code>login</code>	43
18.5	5.7.7 Visualizzazione Setuid e Setgid Bits	43
19	5.8 Role-Based (RBAC) e Mandatory Access Control	44
19.1	5.8.1 Mandatory e Discretionary Access Control	44
19.2	5.8.2 Role-Based Access Control (RBAC)	44
	19.2.1 Esempio RBAC	45
19.3	5.8.3 M-AC e SELinux	45

20	5.9 OAuth 2.0 – Open Authorization	46
20.1	5.9.1 Introduzione	46
20.2	5.9.2 Principi di OAuth 2.0	46
20.3	5.9.3 Ruoli in OAuth 2.0	46
20.4	5.9.4 OAuth 2.0 Scopes	47
20.5	5.9.5 Access Token, Authorization Code e Refresh Token . . .	47
20.6	5.9.6 Funzionamento generale di OAuth 2.0	48
20.7	5.9.7 Grant Types in OAuth 2.0	48
21	Introduzione alla Computer Security Log Management	49
21.1	Definizione di Log	49
21.2	Basi dei Computer Security Logs	50
21.2.1	Security Software Logs	50
21.2.2	2.1.2 Operating System Logs	50
21.2.3	2.1.3 Application Logs	51
21.2.4	2.1.4 Usefulness of Logs	51
21.3	2.2 Necessità della Log Management	51
21.3.1	Normative e Regolamenti	52
21.4	2.3 Sfide nella Log Management	52
21.4.1	2.3.1 Log Generation e Storage	52
21.4.2	2.3.2 Log Protection	52
21.4.3	2.3.3 Log Analysis	53
21.5	2.4 Affrontare le Sfide	53
22	Log Management Infrastructure	54
22.1	Architettura	54
22.2	Funzioni Principali	55
22.2.1	Funzioni Generali	55
22.2.2	Funzioni di Storage	55
22.2.3	Funzioni di Analisi	56
22.2.4	Funzioni di Disposal	56
22.3	Syslog-Based Centralized Logging Software	56
22.3.1	Formato Syslog	56
22.3.2	Syslog Security	57
22.4	Security Information and Event Management (SIEM)	57
22.5	Software Aggiuntivi	58
23	Race conditions e risoluzione dei nomi di file a risorse	59
23.0.1	TOCTOU (Time-Of-Check → Time-Of-Use)	59
23.0.2	Perché è difficile rendere “safe” un pathname	59
23.0.3	Esempi pratici di exploit	60

23.0.4	Mitigazioni e pratiche sicure (schematiche)	60
23.0.5	Tecniche specifiche consigliate	61
23.0.6	Rischi residui e considerazioni pratiche	62
23.0.7	Riepilogo rapido (bullet)	62
24	Smashing the stack, stack frames e controllo del <i>instruction pointer</i>	63
24.0.1	Concetti di base sullo <i>stack</i> e sullo <i>stack frame</i>	63
24.0.2	Elementi principali:	64
24.0.3	Buffer overflow: effetto e conseguenze (conciso)	64
24.1	Shell vulnerability	67
24.1.1	Misure di mitigazione (panoramica tecnica ma non operativa)	68
24.1.2	Evoluzioni dell'exploitation (breve)	69
25	Encryption and Decryption (Concetti generali)	70
25.1	Definizione di cifratura	70
25.2	Plaintext e Ciphertext	70
25.3	Principi	70
25.4	Modello generale	70
25.5	Exhaustive Key Search	71
25.6	Esempio: DES	71
25.7	Modelli di attacco	71
25.8	Tipi di avversari	71
26	Symmetric-Key Encryption and Decryption	71
26.1	Definizione e Logica Simmetrica	71
26.2	Cifratura a Bit (Bit Cipher) e Stream Ciphers	72
26.3	Cifratura a Blocchi (Block Ciphers)	72
26.4	AES (Advanced Encryption Standard)	73
26.5	Integrità e MAC (Message Authentication Code)	73
27	Crittografia Asimmetrica	73
27.1	Funzionamento	74
27.2	Proprietà principali	74
28	2.5 Funzioni di hash crittografiche (Cryptographic hash functions)	74
28.1	Definizione e scopo	74
28.2	Proprietà fondamentali	75
28.3	Proprietà pratiche	75

28.4 Esempi e famiglia comuni	76
28.5 Applicazioni principali	76
28.6 Birthday paradox (motivazione collision-resistance)	76
28.7 Uso corretto con firme digitali	76
28.8 Esempio sintetico: verifica integrità file	76
28.9 Note pratiche su password hashing	77
29 2.6 Message Authentication (Data Origin Authentication)	77
29.1 Obiettivo	77
29.2 Messaggi e tag (MAC)	77
29.3 Proprietà desiderate per un MAC	77
29.4 Costruzioni comuni di MAC	77
29.5 MAC vs. Digital Signatures	78
29.6 Freshness e replay protection	78
29.7 Combining MAC e Encryption	78
30 Certificati	78
30.1 Definizione	78
30.2 Funzione	79
31 Certification Authorities (CA)	79
31.1 Ruolo	79
31.2 Benefici	79
32 Lunghezze di Chiave Raccomandate (NIST)	79
32.1 Sicurezza Simmetrica	79
32.2 Equivalenze	79
33 Elliptic Curve Cryptography (ECC)	80
33.1 Concetto	80
33.2 Vantaggi	80
33.3 Svantaggi	80
34 Public-Key Infrastructure (PKI)	80
34.1 Definizione	80
34.2 Scopi	81
35 Validazione della Catena di Certificati	81
35.1 Passaggi di Verifica	81
35.2 Trust Anchors e Certificati Self-Signed	81
36 Estensioni X.509v3	81

37 Cross-Certificates	82
38 Concetti di Fiducia	82
39 Certificate Revocation	82
39.1 Concetto generale	82
39.2 Motivazioni della revoca	82
39.3 Metodi principali di revoca	83
39.3.1 Metodo I — Certificate Revocation Lists (CRL)	83
39.3.2 Metodo II — CRL Fragments (Partitions e Deltas) . .	83
39.3.3 Metodo III — Online Status Checking (OCSP)	83
39.3.4 Metodo IV — Short-Lived Certificates	83
39.3.5 Metodo V — Trusted Public Keys Directly	84
39.4 CA vs End-Entity Revocation	84
39.5 Denial of Service (DoS) e Revoca	84
40 CA/PKI Architectures e Certificate Trust Models	84
40.1 Model IV — Browser Trust Model	84
40.2 Model V — Enterprise PKI Model (Decentralized CA Trust), siti gov o aziende	85
40.3 Model VI — User-Control Trust Model (Web of Trust)	85
40.4 Aspetti chiave	85
41 TLS Website Certificates e Browser Trust Model	86
41.1 Transport Layer Security (TLS)	86
41.2 Gradi di certificati TLS	86

1 Introduzione

La sicurezza informatica riguarda:

- la protezione di **software, computer, reti e informazioni**;
- dispositivi programmabili (*PC, tablet, smartphone*);
- server (*front-end, back-end*, nodi intermedi);
- dispositivi di rete: *firewall, router, switch*.

La sicurezza coinvolge software, canali di comunicazione, interazione umana e possibili abusi.

2 Obiettivi Fondamentali della Sicurezza Informatica

La **sicurezza informatica** è la pratica di proteggere beni informatici da azioni non autorizzate, prevenendole o rilevandole e ripristinando i danni.

2.1 Proprietà fondamentali (Security Properties)

1. **Confidentiality**: le informazioni non pubbliche devono restare accessibili solo a soggetti autorizzati. (serve a sapere chi sei)
Meccanismi: controllo accessi, crittografia, gestione chiavi, restrizioni fisiche.
2. **Integrity**: dati, software e hardware devono rimanere inalterati salvo modifiche autorizzate.
Meccanismi: checksum crittografici, access control, codici di rilevamento errori.
3. **Authorization**: accesso a risorse solo per entità approvate (owner o admin). (una volta che si sa chi sei si procede a vedere cosa puoi fare)
Meccanismi: access control, ACL, ruoli utente.
4. **Availability**: risorse e servizi devono essere sempre accessibili agli utenti autorizzati.
Minacce: guasti, attacchi DoS, cancellazioni intenzionali.
5. **Authentication**: garanzia che identità, dati o software siano autentici.
Tipi: entity authentication, data origin authentication.
Supporta: autorizzazione, attribuzione e accountability.

6. **Accountability**: possibilità di identificare chi è responsabile di azioni passate.

Meccanismi: log, transazioni, firme digitali, non-repudiation.

2.2 Terminologia

- **Trusted** = componente di cui ci si fida.
- **Trustworthy** = componente che merita fiducia (mantiene aspettative).
- **Confidentiality vs Privacy**: la privacy riguarda informazioni personali sensibili. (ES: For example, if we put money in a bank, we trust the bank to return it. A bank that has, over 500 years, never failed to return deposits would be considered trustworthy; one that goes bankrupt after ten years may have been trusted, but was not trustworthy.)
- **Anonymity**: azioni non collegate a un'identità pubblica.

3 Politiche di Sicurezza e Attacchi

3.1 Security Policy

ognuno può avere una def diversa per dire che qualcosa è sicuro, la policy infatti ci permette di avere un'idea più concreta
essa definisce:

- chi può accedere a cosa (asset);
- servizi di sicurezza da fornire;
- controlli di sistema.

Una politica può essere formale (stati sicuri/non sicuri, cioè stati in cui la policy è violata) o informale (linee guida).

3.2 Attacchi e Minacce

- **Attack**: sequenza di azioni deliberate per violare la sicurezza.
- **Vulnerability**: debolezza sfruttabile (flaw di design, implementazione o configurazione).
- **Threat agent / Adversary**: entità che può lanciare un attacco.
- **Attack vector**: metodo specifico usato per l'attacco.

3.3 Esempio non tecnico

Politica: nessuno entra in casa senza un familiare.

Vulnerabilità: porta non chiusa.

Attacco: ladro entra e ruba TV.

Attack vector: ingresso dalla porta non chiusa.

4 Rischio e Valutazione del Rischio

4.1 Definizioni

Rischio (Risk) = perdita attesa dovuta a eventi futuri dannosi.

4.2 Equazioni del rischio

sono equivalenti, infatti P riassume T e V.

$$R = T \cdot V \cdot C \quad (1)$$

$$R = P \cdot C \quad (2)$$

Dove:

- T = probabilità che una minaccia si realizzi (threat);
- V = vulnerabilità;
- C = valore dell'asset o costo del danno;
- P = probabilità di un attacco riuscito.

4.3 Expected Loss Modeling

ALE è introdotto per passare dal concetto di rischio astratto (R) a una misura economica concreta dato che considera l'arco temporale

$$ALE = \sum_{i=1}^n F_i \cdot C_i$$

Dove:

- F_i = frequenza annuale di eventi di tipo i ;
- C_i = perdita media per evento.

4.4 Domande chiave per la valutazione del rischio

1. Quali asset sono più preziosi?
2. Quali vulnerabilità esistono?
3. Quali threat agent e attack vector sono rilevanti?
4. Qual è la probabilità/frequenza d'attacco?

4.5 Cost-Benefit Analysis

Principio: il costo di una misura difensiva non deve superare il beneficio (riduzione delle perdite).

4.6 Valutazione qualitativa

Usa categorie (molto basso → molto alto) invece di valori numerici (da 1 a 10).

Esempio: Risk Rating Matrix (valori da 1 a 5 basati su probabilità e impatto, matrice che tiene in considerazione sia il valore della risorsa sia la probabilità di attacco riuscito).

4.7 Gestione del rischio

Opzioni:

- mitigazione tecnica o procedurale;
- trasferimento (assicurazione);
- accettazione (se il costo della difesa è maggiore);
- eliminazione (disattivazione sistema).

5 Modellazione dell'Avversario e Analisi della Sicurezza

5.1 Attributi di un adversary

1. Obiettivi (target assets);
2. Metodi (attack techniques);

3. Capacità (risorse, competenze, accesso);
4. Finanziamento;
5. Insider vs Outsider.

5.2 Classificazione degli avversari (Named Groups)

- intelligence straniera;
- cyber-terroristi;
- spionaggio industriale;
- crimine organizzato;
- cracker (Il cracker è chi viola intenzionalmente la sicurezza di un sistema per danneggiarlo, trarne profitto o aggirare protezioni. l'hacker è chi ha grandi conoscenze del sistema) o piccoli criminali;
- insider malevoli;
- dipendenti non malevoli ma negligenti.

5.3 Schemi di classificazione

- Per **capacità e intenzione** (Livello 1–4);
- Per **targeting**: attacchi mirati o opportunistici.

5.4 Security Evaluation e Penetration Testing

- **Security Evaluation**: certificazione formale da enti terzi.
- **Penetration Testing (Pen Test)**: serve a valutare la sicurezza del sistema e ci sono tre tipi:
 - *black-box*: senza accesso a codice o documenti;
 - *white-box*: con informazioni interne;
 - *grey-box*: parziale accesso alle informazioni;
 - scopo: individuare vulnerabilità tramite exploit controllati.

5.5 Security Analysis

- Scopo: identificare vulnerabilità e minacce non considerate.
- Deve iniziare presto nel ciclo di vita del software.
- Include: code review, analisi del design, verifica dei requisiti.
- Output: elenco minacce mitigate e non mitigate.

5.6 Security Models

Modelli astratti che collegano componenti di sistema alle politiche di sicurezza. Utili per verificare il rispetto dei requisiti di sicurezza.

6 Threat Modeling: Diagrams, Trees, Lists and STRIDE

6.1 Definizione generale

Il **threat modeling** identifica *threats*, *threat agents* e *attack vectors* che un sistema considera in scope per difendersi, siano essi noti o anticipati. Elementi out-of-scope devono essere registrati esplicitamente. Considera inoltre *adversary modeling* e tutte le assunzioni sul sistema, l'ambiente e gli attaccanti.

6.2 Approcci principali

I principali approcci per il threat modeling includono:

- Diagram-driven
- Attack trees
- Checklists
- STRIDE

6.3 Diagram-driven Threat Modeling

- Partire da un *architectural diagram* del sistema.
- Evidenziare componenti, *data flows* e *system gateways*.
- Delimitare i *trust domains* (es. server autenticati, percorsi filtrati da firewall).
- Trasformare il diagramma in *data flow diagram* informale, tracciando il flusso di dati per task rappresentativi.
- Considerare il *user workflow* e il ciclo di vita dei dati, software e account.
- Verificare posizioni di dati sensibili e percorsi di accesso autorizzati.
- Analizzare tutte le comunicazioni (wireline e wireless) e possibili scenari di attacco.
- L'obiettivo è stimolare brainstorming strutturato e libero sui potenziali threat e attack vectors.

6.4 Attack Trees

- Radice: *overall attack goal* (obiettivo raggiunto es: entrare in casa).
- Nodi figli: alternative per raggiungere l'obiettivo (OR: basta uno dei due per risalire al padre AND: necessario combinare più step).
- Foglie: azioni finali dell'attacco (usare ddos).
- Ogni path radice-foglia rappresenta un *attack vector*.
- Annotazioni: fattibilità, costi, priorità, tipologia di avversario.
- Utili per:
 - Formare *security policy*
 - Analizzare se i meccanismi difensivi coprono tutti gli attack vectors
 - Stimolare il pensiero creativo (think like an attacker)

6.5 Checklists e STRIDE

Checklists: liste di attacchi noti, usate come riferimento o in approcci ibridi; vantaggi: completezza, apprendimento; svantaggi: generalizzazione, rischio di trascurare minacce specifiche.

STRIDE: acronimo per sei categorie di threat:

- Spoofing — impersonazione di entità
- Tampering — alterazione non autorizzata
- Repudiation — negazione di responsabilità
- Information Disclosure — divulgazione dati non autorizzata
- Denial of Service — impatto su disponibilità o qualità
- Escalation of Privilege — escalation privilegi

STRIDE aiuta a stimolare il pensiero aperto durante l'analisi dei diagrammi.

7 Model-Reality Gaps and Real-World Outcomes

7.1 Difficoltà del Threat Modeling

- **Gap modello-realtà:** discrepanze tra modello astratto e realtà tecnica possono dare falsa sicurezza.
- Principali errori:
 1. *Invalid assumptions* (trust errato)
 2. Focus su *wrong threats*
- Cause: cambiamenti tecnologici, nuove vulnerabilità, informazioni incomplete o astratte.

7.2 Esempi pratici

- *Hotel safebox:* assunzione errata su manutenzione e staff.
- *Online trading fraud:* difese limitate a trasferimenti diretti ignorano manipolazioni di mercato.

- *Phishing one-time passwords*: inganno dell'utente per ottenere password temporanee.
- *Bypassing perimeter defenses*: USB malevoli, accesso diretto al sistema.

7.3 Iterative Process: Evolving Threat Models

- Il threat modeling è iterativo, da aggiornare con nuove informazioni e attacchi.
- Esempio: protocolli Internet originari assumono endpoint affidabili, oggi spesso compromessi da malware.
- Considerare sia attacchi software sia hardware (*hard e soft keyloggers*).

7.4 Security Policy e Real-World Outcomes

- Difese devono supportare *security policy*.
- Outcome possibili:
 1. Difese insufficienti → policy violata
 2. Difese corrette, policy completa → sistema “secure”
 3. Policy incompleta → sicurezza formale vs aspettative reali

7.5 Security Analysis: Key Questions

- Quali asset sono preziosi (*protection goals*)?
- Quali attacchi li mettono a rischio?
- Come fermare o gestire azioni dannose?
- Mitigazione: prevention, detection, real-time response, recovery, insurance.

7.6 Testing e Assurance

- Il testing è incompleto: impossibile prevedere tutti gli attacchi.
- Sicurezza non osservabile direttamente (*negative goal*): assenza di flaw non garantita.
- Assurance parziale: miglioramento iterativo, analisi ad hoc, modeling formale, esperienza, lezione dagli attacchi.

8 1.7 Design principles for computer security

8.1 Prefazione

Non esiste una checklist completa che garantisca la *security*. Tuttavia, esistono principi di progettazione largamente applicabili che riducono errori, superficie d'attacco e complessità; qui sono elencati e commentati.

8.2 Principi (P1–P20)

- P1 SIMPLICITY-AND-NECESSITY:** mantenere design semplici e minimali; disabilitare funzionalità non essenziali; ridurre il numero di componenti e interfacce per minimizzare l'*attack surface*.
- P2 SAFE-DEFAULTS:** impostazioni sicure di default; *deny-by-default*; preferire *whitelist* a *blacklist*; *fail-safe* (se fallisci chiudi l'accesso).
- P3 OPEN-DESIGN:** evitare *security by obscurity*; favorire revisione pubblica; non dipendere dal segreto del design, se tutti possono craccare il codice allora sai renderlo più robusto più in fretta (Kerckhoffs' principle).
- P4 COMPLETE-MEDIATION:** ogni accesso deve essere mediato e verificato (autenticazione + autorizzazione + integrità della richiesta).
- P5 ISOLATED-COMPARTMENTS:** compartmentalizzare con forte isolamento (processi, VM, sandbox, zone, firewall) per limitare impatto di guasti e escalation di privilegi.
- P6 LEAST-PRIVILEGE:** assegnare il minimo privilegio necessario e per il minor tempo possibile; ridurre uso di privilegi di superuser.
- P7 MODULAR-DESIGN:** evitare moduli monolitici con privilegi concentrati; favorire granularità e separazione di ruoli (separation of duties, thresholding).
- P8 SMALL-TRUSTED-BASES:** minimizzare codice fidato (*trusted computing base*), facilitando analisi e verifica.
- P9 TIME-TESTED-TOOLS:** riutilizzare tool e primitive consolidate (es. librerie crittografiche ben esaminate) invece di implementare ad hoc.
- P10 LEAST-SURPRISE:** interfacce e comportamenti coerenti con le aspettative utente per ridurre errori umani (user-friendly).

- P11 USER-BUY-IN:** progettare meccanismi che gli utenti vogliano usare (path of least resistance deve essere anche sicuro).
- P12 SUFFICIENT-WORK-FACTOR:** la difficoltà di rompere una difesa deve superare le risorse previste degli attaccanti (es. lunghezza chiavi, entropia password).
- P13 DEFENSE-IN-DEPTH:** difese multiple e indipendenti; evitare punti singoli di fallimento; rafforzare l'anello più debole.
- P14 EVIDENCE-PRODUCTION:** registrare eventi (log, audit trail) per accountability, detection e forensic.
- P15 DATA-TYPE-VERIFICATION:** validare che i dati ricevuti rispettino il formato atteso; sanitizzare e canonicalizzare input per prevenire injection.
- P16 REMNANT-REMOVAL:** rimuovere tracce sensibili a sessione terminata (chiavi, dati in RAM, cache, spazio secondario).
- P17 TRUST-ANCHOR-JUSTIFICATION:** giustificare e verificare i punti di fiducia (trust anchors); cura particolare a registrazione/inizializzazione.
- P18 INDEPENDENT-CONFIRMATION:** usare canali/controlli indipendenti per confermare integrità o autenticità (es. hash via canale separato).
- P19 REQUEST-RESPONSE-INTEGRITY:** legare risposte alle richieste (binding), verificare coerenza in protocolli distribuiti; usare controlli crittografici di integrità.
- P20 RELUCTANT-ALLOCATION:** non allocare risorse né estendere privilegi verso agenti non autenticati; aumentare onere di prova per chi inizia comunicazioni.

8.3 Principi di alto livello e massima

- HP1 SECURITY-BY-DESIGN:** inserire security fin dalle fasi iniziali di design; dichiarare obiettivi e assunzioni di sicurezza.
- HP2 DESIGN-FOR-EVOLUTION:** progettare per aggiornamenti e *algorithm agility*; predisporre processi di re-evaluation continuativa.
- Maxim VERIFY FIRST:** meglio verificare prima di fidarsi (principi correlati: P4, P15, P17, P18).

9 Capitolo 3: User Authentication — Passwords, Biometrics and Alternatives

9.1 3.1 Password authentication: concetti chiave

- **Definizione:** l'utente fornisce *username* e *password*; il sistema verifica corrispondenza.
- **Limite:** matching non garantisce che chi ha inserito la password sia il legittimo utente (conoscenza non implica esclusività).
- **Storage:** preferire memorizzazione di *hash* ($h_i = H(p_i)$) invece di *cleartext* per evitare esposizione diretta in caso di furto file password.

9.2 Pre-computed dictionary attack (attacco a tavole pre-calcolate)

1. Generare lista candidati $w_1, \dots, w_t$ e memorizzare coppie $(H(w_j), w_j)$ ordinando per hash.
2. Rubare file password con hash $h_i = H(p_i)$.
3. Cercare h_i nella tabella per recuperare p_i se presente.

Conseguenza: hashing senza sale (*salt*) è vulnerabile; salting e iterated hashing mitigano attacco.

9.3 Targeted vs Trawling scope

- **Targeted:** attacco mirato a un utente specifico.
- **Trawling:** attacco su vasta scala per trovare qualsiasi account vulnerabile.

9.4 Approcci per sconfiggere l'autenticazione basata su password

1. **Online guessing:** inviare tentativi al server.
2. **Offline guessing:** possesso del file degli hash permette test illimitati localmente.

3. **Password capture:** keylogger, shoulder-surfing, phishing, middle-person, malware.
4. **Interface bypass:** sfruttare vulnerabilità software per eludere l'interfaccia di autenticazione.
5. **Defeating recovery mechanisms:** compromettere processi di recupero (es. reset via SMS o secret questions).

9.5 Password composition, vantaggi e svantaggi

- **Composition policies:** regole su lunghezza e categorie LUDS (lower, upper, digits, special). Migliorano resistenza a guessing, ma spesso peggiorano usabilità e non proteggono contro capture.
- **Svantaggi:** gestione di molte password, scrittura non sicura, riuso, perdita di usabilità (violazione P11).
- **Vantaggi:** semplici, economiche, onnipresenti; non richiedono hardware aggiuntivo; facili da cambiare; ben comprese.

10 3.2 Password-guessing strategies and defenses

10.1 Online password guessing & rate-limiting

- **Differenza:** online coinvolge il server reale (risposte immediate); difesa comune: *rate-limiting* (throttling), lockout, delay esponenziale.
- **Tradeoff:** bisogna avere un buon compromesso tra limitare i tentativi e invece non limitarli altrimenti si rischia di *DoS* legittimo (utente dimentica la password e viene bloccato); mitigare con recovery sicuri.

10.2 Offline password guessing

- Richiede hash file rubato; numero di tentativi limitato solo da risorse computazionali dell'attaccante.
- Contromisure: *iterated hashing* (fare hashing più volte, serve per rallentare l'attacco) e *salting* (aggiunta di un numero casuale prima della password, pubblico, serve solo a rendere password uguali hashate diversamente impedendo di capire una password e averle tutte, forzando a ricalcolare tutto ogni volta).

10.3 Iterated hashing (password stretching)

- Calcolare $H^d(p)$ (hash iterata d volte) e memorizzare il risultato; aumenta costo per ogni tentativo sia per server che per attaccante.
- Valori pratici: d scelto in funzione di prestazioni server e potenza attaccante (es. $d = 1000$).

10.4 Password salting

- Per ogni utente scegliere un *salt* s_i (t -bit, $t \geq 64$) e memorizzare $(u_i, s_i, H(p_i, s_i))$.
- Rende impraticabili tabelle pre-calcolate globali (aumento lavoro e storage di 2^t).
- Nota: salt pubblico non impedisce attacchi on-the-fly se l'attaccante possiede il file.

10.5 Pepper (secret salt)

- Valore segreto non memorizzato nel file (memorizzato in altri file protetti); sistema prova un insieme di possibili *peppers* durante la verifica.
- Fornisce rallentamento addizionale ma introduce complessità di gestione (se perduto, difficoltà di verifica).

10.6 Specialized password-hashing functions (KDFs)

- Funzioni progettate per essere lente e/o memory-hard (es. Argon2, bcrypt, scrypt, PBKDF2) per ostacolare attacchi GPU/ASIC.
- Migliore scelta rispetto a funzioni hash generiche (MD5, SHA-1) per storage password.

10.7 System-assigned passwords (brute-force) e probabilità di successo

- Spazio password $R = b^n$ (b alfabeto, n lunghezza). Probabilità di successo in periodo T con G tentativi/s:

$$q = \min\left(1, \frac{G \cdot T}{R}\right).$$

- Da qui si ricava la lunghezza minima $n = \frac{\log(R)}{\log(b)}$ per un target q fissato.

10.8 User-chosen passwords e distribuzioni sbilanciate

- Le password scelte dagli utenti hanno distribuzioni fortemente skewed (sbilanciata, prevedibile e non casuale): poche password molto popolari.
- Difesa efficace: *blacklisting* (pro-active password checking) di password comuni; auto-cracking interno per notificare utenti deboli.

10.9 Login passwords vs passkeys

- **Login web**: composizione complessa spesso sostituita da rate-limiting, blacklisting, salting, iterated hashing (nel web login moderno non serve complicare eccessivamente la password, ma potenziare la protezione lato server contro attacchi automatici).
- **Passkeys / key derivation keys**: per cifratura di dati a lungo termine richiedere maggiore entropia e resistenza offline (usable con password manager, deve essere una password molto complessa).

10.10 Tabella riassuntiva (misure vs attacchi)

- **rate-limiting** — mitiga *online guessing* (significa limitare i tentativi, attenzione a logout).
- **blacklisting** — riduce successo di guessing su password popolari.
- **salt** — mitiga attacchi pre-computati (rainbow tables).
- **iterated hashing / KDF** — rallenta *offline guessing*.
- **pepper / MAC** — aumenta difesa offline (chiave segreta separata).

11 3.3 Account recovery and secret questions

11.1 Meccanismi comuni di recovery

- **Recovery email / recovery link**: invio link o codice temporaneo all'indirizzo secondario registrato.
- **Code via telecom device (SMS)**: invio OTP a numero registrato (dipende binding numero-dispositivo).

- **Secret questions (challenge questions):** risposte memorizzate che permettono reset; problema: spazio delle risposte piccolo e spesso pubblico.

11.2 Problemi di sicurezza e usabilità

- **Usabilità:** risposte cambiano nel tempo, utenti dimenticano, utenti inventano risposte false.
- **Sicurezza:** risposte facilmente indovinabili o reperibili online; spesso memorizzate in chiaro sul server (rischio privacy).
- **Conclusione:** secret questions sono deboli; preferire canali indipendenti (email secondaria, hardware token) o combinare più fattori.

11.3 Esempio: password reset attack (interleaving)

- Attacco che sfrutta sito malizioso A per raccogliere risposte e codici e in tempo reale usarli per resettare account su sito B; dimostrato in pratica contro servizi reali.
- Implicazione: processi di recovery che si affidano solo a SMS o secret questions sono vulnerabili.

12 3.4 One-time password generators and hardware tokens

12.1 Motivazione

- Password statiche possono essere riprodotte da attaccanti che le catturano; gli OTP sono validi una sola volta, riducendo rischio di replay.

12.2 Modalità comuni di OTP

- **Paper lists:** liste cartacee fornite dalla banca; server mantiene corrispondente lista.
- **SMS OTP:** codice inviato via SMS al numero registrato (vulnerabile a *SIM swap*).
- **Hardware tokens / app OTP:** dispositivi o app (TOTP/HOTP) generano codici basati su tempo o contatore.

12.3 Vettori d'attacco specifici: SIM swap

- Attaccante convince provider mobile a trasferire numero su SIM controllata; poi riceve SMS OTP destinati alla vittima.

12.4 Passcode Generators

- Devices (or smartphone apps) store user-specific secret s_A and compute OTP using a TVP challenge.
- TVP can be explicit (user-entered string) or implicit (time-based).
- Used as a second factor alongside static passwords.
- Advantages: avoids interception risks like SMS OTPs.

12.5 Hardware Tokens

- "What you have" authentication: USB keys, smart cards, mobile devices.
- Example: USB token holds RSA key, responds to challenge r_B with r_A and signature $S_A(r_A, r_B)$.
- Authenticator: any hardware/software producing secret-based strings for authentication.

12.6 User Authentication Categories

- **Knowledge-based ("what you know")**: passwords, PINs, passphrases.
- **Possession-based ("what you have")**: hardware tokens, devices with cryptographic secrets.
- **Biometrics ("what you are")**: physical (fingerprints, iris) or behavioral (typing patterns, gait).
- **Location-based ("where you are")**: geolocation of user device.

12.7 Multi-Factor Authentication (MFA)

- Combines multiple categories; two-factor authentication (2FA) is simplest form.
- Goal: independent protection; compromise of one factor should not defeat the other.
- Complementary factors: avoid redundancy (e.g., two passwords), balance security vs usability.
- Signals: implicit checks (IP address, device fingerprint) can augment authentication silently.

13 Biometric Authentication

13.1 Motivation

- Advantages over passwords: no cognitive burden, scalable, user-friendly.
- Drawbacks: security often overestimated; unsuited for remote authentication without trusted input channel.

13.2 Types of Biometrics

- **Physical (P)**: fingerprints, face, iris, hand geometry, retinal scan.
- **Behavioral (B)**: gait, typing rhythm, mouse patterns.
- **Mixed (M)**: voice authentication.
- Biometric modality: set of features used for authentication (Table ??).

13.3 Enrollment and Verification Process

- **Enrollment**: multiple samples, feature extraction, template creation.
- **Verification**: new sample compared to template, matching score s computed.
- Threshold t : if $s \geq t$, sample accepted as genuine.

13.4 Error Types and Tradeoffs

- **False Reject (FRR)**: legitimate user rejected.
- **False Accept (FAR)**: imposter accepted.
- Higher threshold $\rightarrow$ fewer false accepts, more false rejects (security over usability).
- Lower threshold $\rightarrow$ more false accepts, fewer false rejects (usability over security).
- Equal Error Rate (EER): point where $FAR = FRR$; useful for single-point comparison.
- DET and ROC curves: visualize trade-offs between FRR and FAR.

13.5 Biometric Evaluation Criteria

- **Universality**: do all users have the characteristic?
- **Distinguishability**: are features sufficiently unique across users?
- **Invariance**: are features stable over time?
- **Ease-of-Sampling**: difficulty of acquiring sample.
- **Accuracy**: FAR, FRR, EER, FTE-rate, FTC-rate.
- **Cost**: time, storage, hardware/software expenses.
- **User Acceptance**: privacy, invasiveness, cultural factors.
- **Attack Resistance**: resilience against impersonation, spoofing, substitution, injection.

13.6 Circumvention and Attacks

- Security focus: how easily can imposters bypass the system?
- Methods: system-level generic attacks, modality-specific attacks, liveness detection.

13.7 Authentication vs Identification

- Authentication: user asserts identity; sample matched against corresponding template (one-to-one).
- Identification: no asserted identity; system compares sample against many templates (one-to-many).
- Large user bases: one-to-many matching impractical due to false accept risks.
- Identification suitable for law enforcement or surveillance applications.

14 The Reference Monitor, Access Matrix, and Security Kernel

14.1 Concetto di Reference Monitor

- Introdotto nel 1972 come modello per costruire sistemi sicuri contro utenti malevoli.
- Idea centrale: *ogni riferimento da parte di qualsiasi programma verso dati, programmi o dispositivi deve essere validato contro una lista di tipi di accesso autorizzati*, in base all'utente e/o alla funzione del programma.
- Il **reference monitor** è un modello **subject-object**:
 - **Subject (o principal)**: entità che richiede accesso (utente, processo).
 - **Object**: qualsiasi risorsa o elemento a cui si può accedere (processi attivi, segmenti di memoria, file, periferiche, istruzioni privilegiate, ecc.).

14.2 Access Matrix Model

(poco ottimale dato che si forma una tabella enorme)

- Ogni oggetto ha tipi di accesso definiti (*access attributes*) corrispondenti a permessi o privilegi (es. read, write, execute, delete).
- Per ogni coppia (subject, object) si definiscono i privilegi autorizzati.
- Tutti i privilegi sono rappresentati da una **access matrix**:
 - Righe: subjects i
 - Colonne: objects j
 - Celle $A(i, j)$: **Access Control Entry (ACE)** — insieme di permessi che il subject i ha sull'object j .
- Se $A(i, j) = z$, si dice che i ha un z -access verso j .
- La matrice è **policy-independent**: la politica di accesso dipende dal contenuto delle celle, non dalla struttura.

14.3 Implementazione del Reference Monitor

- È implementato come un insieme di **reference validation mechanisms** software/hardware.
- Ogni richiesta di accesso (i, j, z) è intercettata dal monitor della classe di oggetti di j . (i = soggetto che richiede l'accesso, j = oggetto, z = modalità (read/write..))
- Il monitor controlla se $A(i, j)$ permette z e concede l'accesso solo se autorizzato.
- Realizza il principio P4 (ogni singolo accesso a una risorsa (file, memoria, socket, ecc.) deve sempre passare attraverso un controllo di sicurezza): **complete mediation** (ogni accesso è mediato).

14.4 Requisiti del Reference Validation Mechanism

1. **Tamper-proof**: non modificabile da entità non autorizzate.
2. **Always invoked**: non può essere bypassato.
3. **Verifiable**: sufficientemente piccolo da poter essere analizzato e testato completamente.

14.5 Security Kernel

- È il nucleo del sistema che implementa il reference monitor.
- Contiene le strutture minime per gestire e verificare il controllo d'accesso e funzioni di sicurezza essenziali.
- Difficile da implementare pienamente; i sistemi operativi comuni (es. Unix) restano monolitici, violando principi P7 (modular design) e P8 (small trusted bases).
- Ha influenzato profondamente la ricerca sulla sicurezza e l'uso di **access control lists (ACLs)**.

14.6 Dipendenze del Reference Monitor

- Necessita di:
 - Sistema di **autenticazione affidabile**.

- Hardware correttamente funzionante e fisicamente protetto.
- Sicurezza dei **canali I/O** e dei dispositivi di memoria.
- Catena di produzione e fornitura **trustworthy**.
- L'integrazione di componenti da fornitori globali rende difficile garantire l'affidabilità complessiva.

14.7 Implementazione dell'Access Matrix

- L'access matrix è un modello concettuale; l'implementazione diretta è inefficiente (matrice grande e sparsa).
- In pratica, i permessi vengono memorizzati in liste:
 - Per righe → **Capability Lists (C-lists)**.
 - Per colonne → **Access Control Lists (ACLs)**.

14.8 Capability Lists (C-Lists)

- Si concentrano sul singolo subject i^* .
- Contengono coppie $(j, A(i^*, j))$ per ogni oggetto accessibile da i^* .
- Ogni voce specifica i permessi del subject su ciascun oggetto.
- Le C-lists possono essere conservate o associate ai soggetti, e usate quando richiesto.

14.9 Access Control Lists (ACLs)

- Si concentrano sul singolo oggetto j^* .
- Contengono coppie $(i, A(i, j^*))$ per tutti i subject autorizzati ad accedere a j^* .
- Possono essere memorizzate insieme all'oggetto.
- Ogni ACE include campi:
 - (subject; access-type-id = permission-bits).
- Esempio: (adamjones; RWX = 001).
- Permessi possono essere assegnati anche a **gruppi di utenti** (Unix-like).

14.10 Capability-Based vs ID-Based Protection

- **Capability-based (ticket-oriented):**
 - Ogni subject possiede *capabilities* o *tokens* che concedono accesso.
 - Non dipende dall'identità, ma dall'autenticità del token.
 - Analogo a un biglietto valido per l'ingresso a un evento.
- **ID-based:**
 - Il controllo si basa sull'identità del richiedente (foto, credenziali, account).
 - L'oggetto mantiene una lista di identità autorizzate.
- Capabilities devono essere **unforgeable**; le identità **unspoofable**.

14.11 Implementazione delle Capability Systems

- Obiettivo: prevenire la copia o modifica non autorizzata delle capabilities.
- Possibili approcci:
 1. Memorizzazione delle capabilities in memoria kernel.
 2. Memorizzazione in memoria utente, protetta con **Message Authentication Code (MAC)**.
 3. Supporto hardware: **tag bits** gestiti dal kernel per marcare le parole di memoria che contengono capabilities.

14.12 Access Control e Audit Trails

- Ogni accesso mediato dal reference monitor può essere registrato in **audit logs**.
- Scopi:
 - Debugging, accountability.
 - Intrusion detection e forensic investigation.
- I log devono essere **tamper-proof** se usati a fini legali o investigativi.

14.13 Access Control tramite Encryption e Key Release

- Accesso ai documenti può essere gestito tramite **web server** e **crittografia**.
- Meccanismo:
 - I documenti sono cifrati; le chiavi sono rilasciate solo a soggetti autorizzati.
 - Ogni accesso genera un record di audit che indica orario e identità dell'utente.
- Permette di coniugare **access control** e **accountability**.

ad oggi si usa prevalentemente il metodo di accesso basato su Owner/-group/Others dato che permette di ottimizzare lo spazio per memorizzare le regole degli accessi

15 5.4 Implementation Issues

15.1 5.4.1 Aspetti generali

- Oltre alla rappresentazione della **Access Control Matrix** (tramite ACLs o Capabilities), è necessario considerare:
 - Come rappresentare i **subjects** (utenti o processi) all'interno del computer.
 - Come rappresentare gli **objects** (risorse, file, dispositivi, ecc.).
 - Come rappresentare i **diritti di accesso** (access rights) all'interno di una ACL o di una Capability.

15.2 5.4.2 Subjects: Users

- Gli **user accounts** sono memorizzati nel file `/etc/passwd`.
- L'autenticazione avviene tramite password salvata in `/etc/shadow`.
- Formato di un account utente:

```
username:password:UID:GID:name:homedir:shell
```

- Dopo l'autenticazione, il sistema genera un **processo** con l'UID e il GID dell'utente (numeri a 16 bit che identificano univocamente l'utente nel sistema).
- Tutti i processi generati da quell'utente ereditano il suo UID.

15.3 5.4.3 Superuser

- Il **superuser** (amministratore) è un principal privilegiato con `UID = 0` e username solitamente `root`.
- Poche restrizioni:
 - Tutti i controlli di sicurezza sono disabilitati.
 - Può diventare qualsiasi altro utente.
 - Può modificare l'orologio di sistema.
 - Non può scrivere su filesystem di sola lettura, ma può rimontarli in modalità scrivibile.
 - Non può decriptare password ma può reimpostarle.

15.4 5.4.4 Groups

- Gli utenti appartengono a uno o più **gruppi**.
- Il file `/etc/group` contiene tutti i gruppi:

```
groupname:password:GID:list of users
```

- Ogni utente ha un gruppo primario (GID memorizzato in `/etc/passwd`).
- I gruppi forniscono una base comoda per decisioni di access control.
- File principali:
 - `/etc/passwd`: informazioni sugli account utente.
 - `/etc/shadow`: password cifrate.
 - `/etc/group`: informazioni sui gruppi.

15.5 5.4.5 Assegnazione UID ai processi

- Gli utenti lanciano comandi o programmi, eseguiti dal sistema operativo.
- Ogni programma genera uno o più **processi**.
- Un processo è composto da:
 - Codice del programma.
 - CPU.
 - Memoria.
 - Risorse di I/O.
- I processi sono indipendenti, con risorse private.

15.6 5.4.6 Creazione dei processi

- Un processo può essere creato:
 - Su richiesta esplicita dell'utente.
 - Su richiesta esplicita di un altro processo (`fork()`).
 - Durante l'inizializzazione del sistema.

- Tramite routine di sistema.
- Il **Process Control Block (PCB)** contiene il contesto del processo.
- Spazio di indirizzamento virtuale tipico di ogni processo (come è strutturato):
 - sezione per il Program code, Contiene le istruzioni eseguibili del programma (text).
 - Variabili globali e statiche (Static data).
 - allocazione dinamica (Heap).
 - allocazione dinamica (Stack).

15.7 5.4.7 System Call **fork()**

- La **fork()** crea una copia del processo chiamante.
- Dopo la **fork()**, esistono due processi:
 - Il processo padre.
 - Il processo figlio (child), copia esatta del padre.
- Il figlio eredita:
 - Codice, stack, heap, file descriptor, variabili globali, program counter.
 - UID del genitore.
- Il figlio riceve nuovo PID, segnali e timestamp propri.
- **Valori di ritorno:**
 - -1 in caso di errore.
 - 0 al processo figlio.
 - PID del figlio al processo padre ES CODICE DELLE SLIDE: [1002](pid di chi ha fatto la fork) [0](pid del figlio (se 0 allora siamo già nel figlio)) i=1(iterazione).

15.8 5.4.8 Objects in UNIX

- Tutte le risorse (file, directory, dispositivi di memoria, I/O) sono trattate come **file**.
- Il controllo degli accessi è quindi ridotto a quello del **file system**.
- Le risorse sono organizzate in una struttura ad albero.
- Ogni voce di file in una directory punta a un'informazione detta **i-node**.

15.9 5.4.9 Objects in Windows

- Windows adotta una modellazione a oggetti per le risorse di sistema. (risulta molto più complesso e per questo linux è più semplice)
- Tipi principali:
 - **File Object**: file o directory su un dispositivo di storage.
 - **Directory Object**: cartelle nel file system.
 - **Process Object**: applicazioni o task in esecuzione.
 - **Thread Object**: unità di esecuzione all'interno di un processo.
 - **Window Object**: elementi GUI (finestre, bottoni, etichette).
 - **Device Object**: dispositivi hardware (stampanti, dischi, rete).
 - **Event / Mutex / Semaphore**: sincronizzazione e mutua esclusione.
 - **File Mapping Object**: mappatura di file in memoria.
 - **Registry Key / Value Object**: chiavi e valori del Registro di sistema.
 - **Security Descriptor Object**: definisce le ACL per un oggetto.
 - **Job Object**: gruppo di processi gestiti insieme.
 - **Pipe / Socket Object**: canali di comunicazione tra processi o su rete.

16 5.5 Privilege Escalation

16.1 5.5.1 Definizione

- È una funzione che permette a utenti autorizzati di elevare temporaneamente i propri privilegi per svolgere attività che richiedono permessi superiori.

- Esempio: installare aggiornamenti o risolvere problemi di sistema.
- Si distingue tra:
 - **Authorized privilege escalation**: controllata e legittima.
 - **Unauthorized privilege escalation**: malevola, sfrutta vulnerabilità.
- Nei sistemi Unix-like l'elevazione controllata è implementata tramite **Set-UID**, basato sul principio di **Complete Mediation**.

16.2 5.5.2 Password Dilemma

- Il file `/etc/shadow` ha permessi ristretti.
- Gli utenti normali non possono modificare la propria password direttamente.
- Soluzione: usare programmi privilegiati come `passwd` con **Set-UID**.

16.3 5.5.3 Set-UID Concept

- Permette a un utente di eseguire un programma con i privilegi del suo proprietario.
- Esempio:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 41284 Sep 12 2012 /usr/bin/passwd
```

- Il bit “**s**” indica che il programma è eseguibile con **privilegi elevati**, **S** maiuscolo che ha i privilegi elevati ma non è eseguibile, vale anche per l’x del group e per l’x del other (per quest’ultimo però è **t** o **T** e indica lo sticky che Blocca cancellazione/modifica da altri utenti in directory condivise).

16.4 5.5.4 Implementazione Set-UID

- Ogni processo ha tre User ID:
 - **RUID** (Real UID): identificazione primaria del processo.
 - **EUID** (Effective UID): determina i permessi effettivi di accesso.

- **SUID** (Saved UID): copia di sicurezza del RUID.
- Esecuzione normale: $RUID = EUID$.
- Esecuzione Set-UID: $RUID \neq EUID$.
- Se il programma è di proprietà di **root**, viene eseguito con privilegi **root**.

16.5 5.5.5 Come abilitare Set-UID

- Impostare il proprietario del file a **root**.
- Abilitare il bit Set-UID:

```
chmod u+s /path/to/file
```

- Rimuoverlo:

```
chmod u-s /path/to/file
```

- In forma ottale:

```
chmod 4777 /path/to/file
```

(Il prefisso 4 indica il bit Set-UID).

16.6 5.5.6 Funzionamento con `fork()` e `exec()`

- Quando un processo p viene creato tramite `fork()`, eredita tutti gli UID del genitore.
- Quando p invoca `exec(PROG)`:
 - Se **PROG** ha il bit Set-UID, $EUID$ diventa l'UID del proprietario del programma.
 - Se non lo ha, $EUID = RUID$.

17 5.6 Object Permissions e File-Based Access Control

17.1 5.6.3 ACL Alternatives

- Sistemi moderni supportano ACL per oggetti di sistema, inclusi file.
- Vantaggi: precisione fine-grained.
- Svantaggi: memoria e tempi di ricerca, aggiornamenti frequenti, liste lunghe.
- Alternativa storica: modello **ugo** (user-group-others), meno espressivo ma efficiente.
- Altra alternativa: Role-Based Access Control (Section 5.7).

17.2 5.6.4 File Owner e Group

- Ogni file ha un **owner** (UID) e un **protection group** (GID).
- Valori iniziali derivano dal processo che crea il file.
- UID e GID collegati a `/etc/passwd` e `/etc/group`.
- Password storicamente in `/etc/passwd`, oggi in `/etc/shadow`.

17.3 5.6.5 Superuser vs Root

- Il controllo degli accessi si basa su UID, non su username.
- Superuser: UID=0, accesso a tutte le risorse file indipendentemente dai permessi.
- Username `root` è convenzione, UID=0 determina i privilegi.

17.4 5.6.6 User-Group-Others (ugo) Model

- Tre categorie di permessi:
 - **User**: proprietario del file.
 - **Group**: condivisione tra piccoli gruppi di utenti.
 - **Others**: tutti gli altri utenti.

- Permessi codificati in bit, efficienti in storage e processing.
- Limite: perdita di espressività rispetto alle ACL.

17.5 5.6.7 Meta-data e File Permissions

- Ogni file contiene una struttura dati che memorizza:
 1. **user**: UID proprietario.
 2. **group**: GID gruppo.
 3. 9 bit: 3 permessi (R,W,X) per user, group, others.
 4. 3 bit speciali: setuid, setgid, t-bit.
- Significato permessi per file regolari:
 - R (read): leggere contenuto.
 - W (write): modificare contenuto.
 - X (execute): eseguire file binario o shell script.

17.6 5.6.8 Uso dei Protection Bits

- Quando un processo richiede accesso:
 1. Controllo sequenziale: user $\rightarrow$ group $\rightarrow$ others.
 2. La prima categoria qualificante determina i permessi.
 3. Se user non concede R, la richiesta fallisce anche se others concede R.

17.7 5.6.9 Permission Display Notation

- Formato comune: stringa 10 caratteri, es. `-rwxr-xr-`.
- Primo carattere: tipo file (`-` = file normale, `d` = directory).
- Gruppi di tre caratteri: permessi per user, group, others.
- Bit speciali mostrati modificando 'x' in 's', 'S', 't', 'T' (Fig. 5.5).

17.8 5.6.10 Valori iniziali dei Protection Bits

- 9 bit RWX impostati alla creazione del file tramite **mode parameter** della syscall.
- Modificati dalla **umask** dell'utente.
- Esempio valori comuni:
 - Default 666 (RW per tutti)
 - Umask 022 (rimuove W per group e others)
 - Risultato combinato 644: RW per user, R per group e others.

18 5.7 Setuid Bit e Effective UserID (eUID)

18.1 5.7.3 Setgid Bit

- Analogamente a setuid, ma applicato al **protection group** del file.
- Valori tracciati: **rGID**, **eGID**, **sGID**.
- Syscalls di supporto: **getgid()**, **getegid()**, **setgid()**.
- Se directory: setgid permette accesso condiviso nei gruppi (Section 5.5).

18.2 5.7.4 Esempio Setuid: passwd

- Comando Unix **passwd** per cambiare password.
- **/usr/bin/passwd**: root-owned e setuid.
- Permette scrittura sul file password di sistema.
- Controlla rUID per modificare solo la voce dell'utente chiamante.

18.3 5.7.5 Inheriting UserIDs

- Creazione processo via **fork()**:
 - Replica processo padre.
 - Figlio eredita rUID, eUID, sUID del padre.
 - Figlio ottiene nuovo PID.
- Se il figlio esegue un nuovo eseguibile (**exec()**):
 - Eredita PID e userid del figlio, salvo se eseguibile è setuid.

18.4 5.7.6 Esempio UserIDs e login

- `login` esegue `setuid program /bin/login` (root-owned).
- Verifica username-password, imposta UID e GID del processo ai valori dell'utente verificato.
- La shell dell'utente eredita UID e GID del processo `login`.

18.5 5.7.7 Visualizzazione Setuid e Setgid Bits

- Notazione a 10 caratteri:
 - Se setuid o setgid attivo, cambia user/group execute letter:
 - * $x \rightarrow s$
 - * $- \rightarrow S$ (setuid/setgid senza execute, non utile)
 - Esempio:
 - * `-rwsr-xr-x`: eseguibile da tutti, setuid attivo.
 - * `-rwSr-r-`: setuid attivo ma file non eseguibile (non utile).

19 5.8 Role-Based (RBAC) e Mandatory Access Control

19.1 5.8.1 Mandatory e Discretionary Access Control

- Le regole di **access control policy** sono applicate dal sistema operativo.
- **Discretionary Access Control (D-AC)**:
 - Esempi: file permissions in Multics e Unix.
 - Il proprietario della risorsa decide quali permessi concedere ad altri.
- **Mandatory Access Control (M-AC)**:
 - Un security policy administrator definisce, per ogni oggetto, chi può fare cosa.
 - Esempio: multi-level security (MLS) negli USA.
 - Ogni soggetto ha un **security clearance level**.
 - Documenti classificati: Top Secret, Secret, Confidential, Controlled Unclassified, Unclassified.
 - Accesso consentito se clearance del soggetto $\geq$ classificazione del documento.
- Differenza chiave: D-AC è il proprietario del file a decidere, M-AC si basa su policy centralizzate, cioè non è il proprietario a scegliere ma qualcun'altro es: documenti top-secret.

19.2 5.8.2 Role-Based Access Control (RBAC)

- Idea: un **subject** (utente) è assegnato a uno o più **roles** per sessione.
- Ogni ruolo ha un set predefinito di **permissions**.
- Le **permissions** correnti di un soggetto dipendono dai ruoli attivi.
- Riflette gestione dei permessi nelle **larger organizations**.

19.2.1 Esempio RBAC

- Ruolo **GradAdmin**: read/write su file dipartimento (studenti, applicant, budget).
- Ruolo **GrantManager**: read su file grant dipartimento.
- Nuovo membro Alex: assegnati entrambi i ruoli → acquisisce entrambi i set di permessi.
- Sostituzione ruolo: Corey ottiene permessi assegnandosi stessi ruoli.
- Ruoli possono essere **gerarchici**, es. SeniorManager = unione ruoli junior + ruoli specifici.

19.3 5.8.3 M-AC e SELinux

- SELinux è un modulo di sicurezza per Linux che implementa principalmente Mandatory Access Control (M-AC), aggiungendo un livello di controllo molto più rigoroso rispetto ai permessi Unix tradizionali (rwx) o ACL..
- Basato su **Flask OS architecture** e **Linux Security Modules (LSM)**.

20 5.9 OAuth 2.0 – Open Authorization

20.1 5.9.1 Introduzione

- **OAuth 2.0** (Open Authorization) è uno *standard* che consente a un sito o applicazione di accedere a risorse ospitate da altre web app per conto di un utente.
- Sostituisce OAuth 1.0 (2012) ed è oggi lo standard de facto per l'**online authorization**.
- Consente un accesso **consentito e limitato** alle risorse, senza condividere le credenziali dell'utente.
- Utilizzabile non solo su web ma anche su:
 - applicazioni browser-based,
 - applicazioni server-side,
 - app mobili/native,
 - dispositivi connessi (es. smart TV).

20.2 5.9.2 Principi di OAuth 2.0

- OAuth 2.0 è un **authorization protocol**, non un **authentication protocol**.
- Scopo: concedere accesso a risorse (es. API remote, dati utente).
- Utilizza **Access Tokens**:
 - rappresentano l'autorizzazione ad accedere alle risorse per conto dell'utente;
 - formato non definito dallo standard, ma comunemente basato su **JWT (JSON Web Token)**;
 - possono contenere dati interni e hanno una **data di scadenza**.

20.3 5.9.3 Ruoli in OAuth 2.0

- Definiti nella specifica come componenti fondamentali:

Resource Owner L'utente o sistema che possiede le risorse protette e può concedere accesso.

Client Applicazione che richiede l'accesso alle risorse; deve possedere un *Access Token* valido.

Authorization Server – Gestisce le richieste del Client per gli Access Tokens.

- Rilascia token dopo autenticazione e consenso del Resource Owner. (GOOGLE, BANCHE, FACEBOOK, tutte le credenziali e info le detengono loro e non le app che devono verificare l'identità)
- Espone due endpoint:
 - * **Authorization endpoint**: autenticazione e consenso utente.
 - * **Token endpoint**: scambio macchina-macchina (machine-to-machine) prende l'authorization code e restituisce il token o l'eventuale refresh token.

Resource Server Server che custodisce le risorse dell'utente:

- verifica l'Access Token ricevuto dal Client;
- fornisce le risorse autorizzate.

20.4 5.9.4 OAuth 2.0 Scopes

- Gli **Scopes** definiscono il motivo e il livello dell'accesso richiesto.
- Specificano esattamente per quali risorse viene concesso l'accesso.
- I valori ammessi e la loro relazione con le risorse dipendono dal **Resource Server**.

20.5 5.9.5 Access Token, Authorization Code e Refresh Token

- L'Authorization Server può restituire un **Authorization Code** invece di un Access Token diretto (token di breve durata).
- L'Authorization Code viene poi scambiato per un **Access Token**.
- Può essere rilasciato anche un **Refresh Token**:
 - durata lunga (long-lived);
 - può generare nuovi Access Tokens dopo la scadenza;
 - deve essere conservato in modo sicuro dal Client.

20.6 5.9.6 Funzionamento generale di OAuth 2.0

1. Il Client ottiene dal **Authorization Server** le proprie credenziali: *client id* e *client secret*.
2. Il Client invia una **authorization request** con:
 - client id, client secret;
 - scopes richiesti;
 - redirect URI per ricevere Access Token o Authorization Code.
3. L'Authorization Server autentica il Client e verifica gli scopes.
4. Il Resource Owner concede l'accesso (cioè dice solo che il client può accedere ma deve ancora ricevere la chiave (token)).
5. L'Authorization Server reindirizza il Client con un Authorization Code o Access Token (+ eventuale Refresh Token).
6. Il Client usa l'Access Token per richiedere risorse al Resource Server.

20.7 5.9.7 Grant Types in OAuth 2.0

- Ogni **grant type** definisce un flusso specifico per ottenere l'autorizzazione.
- Tipologie principali:

Authorization Code Grant – L'Authorization Server restituisce un *Authorization Code* monouso;

- il Client lo scambia per un *Access Token*;
- ideale per web app tradizionali (scambio sicuro lato server);
- utilizzabile anche da SPA o mobile app, ma senza client secret sicuro.
- Variante sicura: **Authorization Code Grant with PKCE**.

Implicit Grant – Flusso semplificato: Access Token restituito direttamente al Client;

- il token può apparire nel callback URI o come POST form;
- **deprecato** per rischio di leakage del token.

Authorization Code with PKCE (Proof Key for Code Exchange) – Variante sicura per mobile app e SPA;

- introduce codice di verifica (*code verifier*) e challenge (*code challenge*);
- previene attacchi di intercettazione del codice.

Resource Owner Password Credentials Grant – Il Client raccoglie le credenziali dell'utente e le invia all'Authorization Server (l'utente si dovrebbe fidare del client che detiene le sue credenziali e la verifica non avviene nell'authorization server).

- Consentito solo per Client totalmente fidati.
- Nessun redirect: utile quando il redirect non è fattibile.

Client Credentials Grant – Utilizzato da applicazioni non interattive (es. microservizi, processi automatici).

- Il Client si autentica con il proprio client id e secret.

Device Authorization Flow – Specifico per dispositivi con input limitato (es. smart TV);

- consente autorizzazione tramite un secondo dispositivo (es. smartphone).

Refresh Token Grant – Permette di ottenere nuovi Access Token a partire da un Refresh Token valido.

- Evita di richiedere nuovamente il consenso dell'utente.

21 Introduzione alla Computer Security Log Management

21.1 Definizione di Log

Un **log** è un record degli eventi che avvengono all'interno dei sistemi e delle reti di un'organizzazione. È composto da **log entries**, ognuna delle quali descrive un evento specifico. In origine usati per il troubleshooting (processo sistematico di individuazione, analisi e risoluzione di un problema tecnico), oggi i log servono anche per:

- Ottimizzare le prestazioni di sistemi e reti
- Registrare azioni degli utenti
- Fornire dati per indagini su attività malevole

La grande quantità e varietà di log di sicurezza ha portato alla necessità della **log management**, ossia il processo di generazione, trasmissione, memorizzazione, analisi e smaltimento dei dati di log di sicurezza.

21.2 Basi dei Computer Security Logs

I log contengono informazioni su eventi di sicurezza nei sistemi e nelle reti. Le principali categorie sono:

21.2.1 Security Software Logs

Raccolgono dati generati da software di sicurezza:

- **Antimalware Software:** registra malware rilevati, tentativi di disinfezione, quarantene, aggiornamenti e scansioni.
- **Intrusion Detection/Prevention Systems (IDS/IPS):** registrano comportamenti sospetti, attacchi rilevati e azioni intraprese per bloccarli. Alcuni IDS lavorano in modalità batch (es. file integrity checkers).
- **Remote Access Software (VPN):** log di login riusciti/falliti, orari di connessione/disconnessione, dati trasferiti, uso risorse.
- **Web Proxies:** registrano tutte le URL richieste e possono filtrare o proteggere accessi web.
- **Vulnerability Management Software:** registra patch installate, vulnerabilità note, configurazioni host; eseguito periodicamente e genera batch di log.
- **Authentication Servers:** loggano ogni tentativo di autenticazione (origine, utente, esito, data/ora).
- **Routers:** log basici su traffico bloccato o permesso.
- **Firewalls:** log più dettagliati su traffico, stato connessioni, ispezioni contenuti.
- **Network Quarantine Servers:** registrano controlli di sicurezza sugli host e motivi di eventuale quarantena.

21.2.2 2.1.2 Operating System Logs

I sistemi operativi (OS) generano log di eventi legati alla sicurezza:

- **System Events:** azioni operative (es. avvio servizi, shutdown, errori, codici di stato).

- **Audit Records:** autenticazioni, accessi a file, modifiche a policy, creazione/cancellazione account, uso privilegi.

I log OS sono fondamentali per investigare attività sospette su un host e spesso sono in formato **syslog** o in formati proprietari (es. Windows Event Logs).

21.2.3 2.1.3 Application Logs

Applicazioni COTS, GOTS e custom generano log propri o usano quelli dell'OS. Contengono tipicamente:

- **Client Requests e Server Responses:** ricostruzione eventi e sequenze.
- **Account Information:** autenticazioni, modifiche account, privilegi.
- **Usage Information:** numero e dimensioni delle transazioni (utile per monitoraggio o rilevazione anomalie).
- **Operational Actions:** avvii, arresti, errori, cambi di configurazione.

I log applicativi sono molto utili ma spesso in formati proprietari e complessi da interpretare.

21.2.4 2.1.4 Usefulness of Logs

Ogni tipo di log ha utilità diversa:

- Alcuni sono **primari** per rilevare attacchi (es. IDS, firewall).
- Altri sono **secondari**, utili per correlazioni (es. verificare IP ricorrenti).

È fondamentale garantire la **trustworthiness** (affidabilità) delle fonti di log e verificare più sorgenti in caso di compromissione.

21.3 2.2 Necessità della Log Management

La log management:

- Garantisce la conservazione e revisione dei record di sicurezza.
- Aiuta ad identificare incidenti, violazioni, frodi, problemi operativi.
- Supporta auditing, analisi forense, trend analysis e investigazioni interne.

21.3.1 Normative e Regolamenti

Diversi standard richiedono la gestione dei log:

- **FISMA (2002):** impone programmi di sicurezza informatica e controlli sui log.
- **GLBA:** tutela dati clienti nelle istituzioni finanziarie.
- **HIPAA (1996):** impone revisione periodica dei log e conservazione per almeno 6 anni.
- **SOX (2002):** include funzioni IT per audit finanziari e conservazione log.
- **PCI DSS:** richiede il tracciamento degli accessi a dati di carte di credito.

21.4 2.3 Sfide nella Log Management

Le difficoltà principali derivano dal bilanciamento tra risorse e volume dei log.

21.4.1 2.3.1 Log Generation e Storage

- **Molte fonti di log:** OS, applicazioni, dispositivi, ciascuno con più file.
- **Contenuti inconsistenti:** differenze nei campi (magari uno registra ip e data, l'altro nome utente e ora).
- **Timestamps incoerenti:** orologi non sincronizzati rendono difficile la correlazione.
- **Formati diversi:** CSV, tab, syslog, SNMP, XML, binari, ecc.

Soluzione: conversione automatica in formati standard e rappresentazioni uniformi.

21.4.2 2.3.2 Log Protection

I log devono essere protetti per:

- **Confidenzialità e Integrità:** evitare accessi non autorizzati o alterazioni (es. rootkit).
- **Disponibilità:** prevenire sovrascrittura o perdita dati; prevedere archiviazione sicura e filtraggio dei log irrilevanti.

21.4.3 2.3.3 Log Analysis

Problemi principali:

- Analisi trattata come compito a bassa priorità. (le aziende sottovalutano l'importanza di analizzare i log)
- Mancanza di formazione (dipendenti non formati per interpretarli) e strumenti automatizzati (es. SIEM).
- Approccio **reattivo** anziché **proattivo** (si analizzano solo dopo un incidente e non per prevenirlo).

Senza una corretta analisi, il valore dei log è drasticamente ridotto.

21.5 2.4 Affrontare le Sfide

Quattro pratiche chiave per un'efficace gestione:

1. **Dare priorità alla log management:** definire obiettivi, requisiti legali e rischi.
2. **Stabilire policy e procedure:** assicurare coerenza e conformità; condurre audit periodici.
3. **Creare un'infrastruttura sicura:** preservare integrità e confidenzialità dei log, gestire i picchi di dati.
4. **Supportare e formare il personale:** fornire training, strumenti e linee guida tecniche.

22 Log Management Infrastructure

Una **Log Management Infrastructure** è l'insieme di hardware, software, reti e supporti utilizzati per generare, trasmettere, archiviare, analizzare e smaltire i log. Essa si compone di vari elementi e livelli che cooperano per garantire la gestione efficace dei log in un'organizzazione.

22.1 Architettura

Una tipica infrastruttura di log management è suddivisa in tre livelli:

1. Log Generation:

- Include gli *hosts* che generano i log.
- Alcuni host utilizzano *logging clients* che inviano i dati via rete ai log servers.
- Altri host consentono ai server di autenticarsi e recuperare i file di log.

2. Log Analysis and Storage:

- Composto da uno o più *log servers* che ricevono i log in tempo reale o periodicamente.
- I server che raccolgono log da più fonti sono detti *collectors* o *aggregators*.
- I log possono essere memorizzati sui server stessi o in database dedicati.
- Architetture complesse possono includere:
 - Più server con funzioni specifiche (analisi, storage a breve/-lungo termine).
 - Server distribuiti con ridondanza e backup automatico.
 - Due o più livelli di log server (*tiered architecture*) per caching, sicurezza e scalabilità.

3. Log Monitoring:

- Include le *monitoring consoles* per analisi, report e gestione centralizzata.
- I privilegi possono essere limitati per utente e fonte dati.

Considerazioni aggiuntive:

- Le comunicazioni possono avvenire su reti regolari o su reti dedicate per maggiore sicurezza.
- In caso di dispositivi non collegati in rete o legacy, si può effettuare un trasferimento manuale (*out-of-band*) su supporti fisici (es. CD-ROM).
- Gli host con connettività limitata devono comunque essere integrati nel sistema.
- Grandi organizzazioni spesso implementano infrastrutture multiple e indipendenti per motivi di scala, sicurezza o separazione funzionale.

22.2 Funzioni Principali

Le infrastrutture di log management eseguono funzioni per la gestione e l'analisi dei log, senza alterare i log originali.

22.2.1 Funzioni Generali

fatte subito dopo la generazione dei log per renderli leggibili

- **Log Parsing:** estrazione dei valori dai log per ulteriori processi.
- **Event Filtering:** esclusione di eventi irrilevanti o duplicati ES: messaggi di routine (“service started successfully”).
- **Event Aggregation:** unione di eventi simili in un'unica voce con contatore (100 failed login attempts from 192.168.1.10).

22.2.2 Funzioni di Storage

Riguardano la conservazione dei log nel tempo, in modo efficiente e sicuro.

- **Log Rotation:** chiusura e creazione di nuovi file di log secondo orari o dimensioni (una cartella per ogni giorno).
- **Log Archival:** archiviazione a lungo termine su supporti rimovibili, SAN o server dedicati.
 - *Retention:* archiviazione regolare.
 - *Preservation:* conservazione eccezionale per indagini o incidenti.
- **Log Compression:** compressione per risparmiare spazio.
- **Log Reduction:** rimozione di dati non necessari o campi inutili.

- **Log Conversion:** conversione di formato (es. database $\rightarrow$ XML).
- **Log Normalization:** uniformazione dei campi (es. timestamp, time zone).
- **Integrity Checking:** verifica tramite *message digest* (MD5, SHA-1, SHA-256).

22.2.3 Funzioni di Analisi

funzioni per l'analisi effettiva

- **Event Correlation:** correlazione di eventi tramite regole, statistiche o pattern.
- **Log Viewing:** visualizzazione in formato leggibile.
- **Log Reporting:** generazione di report riassuntivi o dettagliati.

22.2.4 Funzioni di Disposal

- **Log Clearing:** eliminazione dei log più vecchi o già archiviati.

22.3 Syslog-Based Centralized Logging Software

Syslog è il protocollo più diffuso per la gestione centralizzata dei log.

22.3.1 Formato Syslog

- Ogni messaggio ha:
 1. **Priority** = *Facility* (da dove viene il messaggio) + *Severity* (quanto è grave l'evento).
 2. **Header:** timestamp + hostname/IP.
 3. **Message body:** contenuto testuale non strutturato.
- **Severity levels:** da 0 (emergency) a 7 (debug).
- Mancanza di standard nei campi interni $\Rightarrow$ analisi automatica difficile.
- Analisi facilitata raggruppando messaggi simili o assegnando codici coerenti.

22.3.2 Syslog Security

Syslog originariamente non prevedeva controlli di sicurezza:

- Trasferimento via **UDP** $\Rightarrow$ non affidabile e senza autenticazione.
- Nessuna cifratura o controllo accessi.
- Vulnerabile a *DoS*, spoofing e sniffing.

RFC 3195 introduce miglioramenti:

- **Reliable Delivery**: uso di **TCP** per affidabilità.
- **TLS Encryption**: protezione confidenzialità in transito.
- **Integrity & Authentication**: uso di **SHA** per message digest.

Estensioni moderne includono:

- Filtraggio avanzato (regex, sorgente, programma).
- Analisi e correlazione log.
- Event response automatica (e-mail, script, SNMP trap).
- Supporto per formati alternativi (es. SNMP).
- Log file encryption e database storage.
- Rate limiting per prevenire DoS.

22.4 Security Information and Event Management (SIEM)

SIEM è una soluzione avanzata e commerciale per la gestione centralizzata dei log, con capacità di analisi e correlazione.

- Può essere:
 - **Agentless**: raccolta diretta via rete.
 - **Agent-Based**: agenti installati su host per filtraggio e normalizzazione locale.
- Analizza, correla e priorizza eventi da diverse fonti.
- Offre:

- GUI per analisti e reportistica.
- *Security Knowledge Base* (vulnerabilità, significato dei log, ecc.).
- *Incident tracking* e workflow.
- *Asset correlation* per priorità degli eventi.
- Comunicazioni sicure via **TCP + encryption**, con autenticazione reciproca.

22.5 Software Addizionali

- **Host-Based IDS:** rileva attività sospette dai log locali (OS, applicazioni, sicurezza).
- **Visualization Tools:** rappresentano graficamente eventi e pattern per facilitare l'analisi.

Conclusione: Una *Log Management Infrastructure* integra generazione, trasmissione, analisi e conservazione dei log, utilizzando protocolli (es. syslog), software avanzati (es. SIEM) e controlli di sicurezza per garantire l'integrità, la disponibilità e la riservatezza delle informazioni registrate.

Hard Link vs Symbolic Link

un file è un percorso di cartelle con un nome dove il contenuto è un puntatore all'area di memoria dove è situato il contenuto del file (inode)

- **Hard link:** crea un nuovo nome che punta direttamente allo *inode* del file originale.
 - Tutti i link condividono lo stesso contenuto.
 - Se il file originale viene cancellato, gli altri hard link rimangono funzionanti.
- **Symbolic link (symlink):** crea un piccolo file contenente il percorso del file originale.
 - Funziona come un "collegamento" o "alias".
 - Se il file originale viene spostato o cancellato, il symlink diventa rotto.

23 Race conditions e risoluzione dei nomi di file a risorse

Premessa Il principio P4 (*COMPLETE-MEDIATION*) richiede che un sistema verifichi l'autorizzazione *prima* di concedere l'accesso a una risorsa — idealmente immediatamente prima dell'uso. Nei sistemi concorrenti questa assunzione si infrange facilmente per condizioni di **race** (TOCTOU: *Time-Of-Check to Time-Of-Use*), in particolare nel contesto dei filesystem e della risoluzione dei pathname. Di seguito un riassunto schematico, completo e compatto, di problemi, esempi e contromisure pratiche.

23.0.1 TOCTOU (Time-Of-Check → Time-Of-Use)

- **Definizione:** controllo di autorizzazione fatto in t_1 , uso dell'oggetto in $t_2 > t_1$; se lo stato può cambiare nell'intervallo, la verifica non garantisce sicurezza.
- **Conseguenza:** un attaccante può modificare il mapping nome→oggetto (es. rimuovere/creare link o file) tra check e use, ottenendo escalation di privilegi o sovrascrittura di risorse protette.
- **Esempio canonico Unix:**
 1. Programma setuid-root P esegue `access("file", PERMS_REQUESTED)` (usa rUID).
 2. Poi fa `open("file", PERMS)` (usa eUID = root) — ma fra i due passi l'attaccante sostituisce "file" con un link a `/etc/passwd`.
 3. Risultato: P apre/sovrascrive `/etc/passwd` come root → **privilege escalation**.
- **Lezione:** non usare `access()` per decisioni di sicurezza quando il successivo operare usa eUID diverso; evitare sequenze `stat/access` → `open` non atomiche.

23.0.2 Perché è difficile rendere “safe” un pathname

- Controllare solo l'inode del file creato in una directory non basta: un avversario che controlli una directory padre può rinominare/sostituire intere dir tree
- **Regola generale:** per garantire che un percorso risolva sempre allo stesso oggetto bisogna avere controllo immutabile (o affidabile) su

TUTTI i nodi della catena `pathname` fino alla `root` — praticamente impossibile in ambienti multi-user senza privilegi kernel o meccanismi atomic filesystem-level.

23.0.3 Esempi pratici di exploit

- **File squatting / symlink attack su `/tmp`:** processo crea file temporanei con nomi prevedibili; attaccante anticipa nome e crea **symlink** verso file sensibile; processo privilegiato apre e sovrascrive la destinazione.
- **Race su directory condivise:** utente A crea `dir/file` in directory che appare sotto controllo, ma un attaccante con permessi su un ancestor(antenato/ cartella padre) può sostituire directory tra check e use.

23.0.4 Mitigazioni e pratiche sicure (schematiche)

1. Preferire operazioni su file descriptor (FD):

- aprire il file con `open()` e poi usare gli equivalenti `fstat()`, `fchmod()`, `fchown()`, `read()/write()` su FD.
- i FD rimangono legati all'inode, quindi sono immuni alla riassegnazione nome→inode successiva.

2. Usare flags atomici del kernel quando disponibili:

- `open(path, O_CREAT | O_EXCL, ...)` per creare file temporanei in modo atomico (fallisce se il file esiste già).
- `openat()`, `fstatat()` con opportuni flag per lavorare relativamente a directory FD (riduce TOCTOU sulle directory).
- `O_NOFOLLOW` per evitare di seguire symlink quando non desiderato.

3. Creazione sicura di file temporanei:

- usare `mkstemp()`, `tmpfile()` o API che ritornano un FD già creato e sicuro (non creare nomi previsti in `/tmp`).
- evitare pattern prevedibili per nomi; non delegare la sicurezza ai permessi di `/tmp` (1777 sticky bit permette sovrascritture da parte del proprietario).

4. Minimizzare finestra di privilegio:

- applicare principio P6 (*LEAST-PRIVILEGE*): saltare ai privilegi elevati solo quando necessario, idealmente usando *drop-privilege-then-fork* o *seteuid()* per eseguire la sola sezione privilegiata.
 - quando possibile, **non** eseguire operazioni sui file con eUID=0 se il file è controllato dall'utente; meglio eseguire `open()` con eUID=rUID e poi comunicare il FD a processo privilegiato in modo sicuro.
5. **Usare rename atomico per sostituzioni:** modifiche al contenuto rispetto a un file esistente sono meglio fatte su file temporanei e poi `rename(temp, target)` (operazione atomica su stessa mount) per evitare stati intermedi osservabili.
 6. **Verifiche post-open (race-resistant checks):**
 - dopo `open()` usare `fstat()` per controllare UID/GID/permessi dell'inode corrente e rilevare cambi di ownership inattesi.
 - se le proprietà non sono quelle attese, chiudere il FD e abortire.
 7. **Limitare l'uso di `access()/stat()` per decisioni di sicurezza:** evitare pattern "check then act" basati su informazioni non atomiche. Se proprio usati, capire rischi e compensazioni.
 8. **Proteggere le directory ascendenti:** se si devono usare pathname sensibili, assicurarsi che directory parent non siano scrivibili da altri; tuttavia questo è spesso impraticabile in ambienti multi-user.
 9. **Usare meccanismi del kernel / capabilities / LSM:**
 - sistemi moderni possono offrire meccanismi di controllo più precisi (capabilities, SELinux, AppArmor) che riducono la necessità di codice `setuid-privilegiato`.
 10. **Logging e auditing:** registrare tentativi falliti e cambi in periodo check→use per rilevare possibili exploitation di TOCTOU.

23.0.5 Tecniche specifiche consigliate

- `mkstemp()` per file temporanei sicuri in directory non condivise.
- `open(path, O_CREAT | O_EXCL | O_NOFOLLOW)` quando si vuole creare se non esiste e rifiutare symlink.
- `openat(dirfd, "name", ...)` con `open()` su directory FD per limitare la superficie di attacco dal path parent.

- uso di `fchown/fchmod` su FD subito dopo l'apertura per impostare permessi in modo non soggetto a TOCTOU.
- evitare di memorizzare decisioni di autorizzazione derivate da `stat()` per usi successivi senza ri-verifica sull'oggetto realmente aperto.

23.0.6 Rischi residui e considerazioni pratiche

- Anche con tutte le contromisure, in ambienti complessi rimane difficile eliminare ogni possibile race se il codice deve operare attraverso namespace condivisi o con componenti non fidate.
- Le soluzioni più robuste spesso richiedono supporto del sistema operativo (syscalls atomiche, controlli del kernel) oppure ridisegno dell'architettura (es. delegare operazioni privilegiate a demoni controllati).
- Portabilità: alcune tecniche (es. comportamento di `setuid()` / `seteuid()`) possono variare tra OS; la soluzione più portabile è basarsi su FD-orientated API e primitive standard (quando disponibili).

23.0.7 Riepilogo rapido (bullet)

- **Problema:** TOCTOU = check non atomico $\Rightarrow$ possibile exploit tramite alterazione nome $\rightarrow$ oggetto.
- **Esempio frequente:** `access()` seguito da `open()` con eUID diverso $\rightarrow$ escalation.
- **Contromisure chiave:** usare FD-based APIs, operazioni atomiche (`O_EXCL`, `openat`), `mkstemp()`, creare+rename atomico, drop-privilege minimo, verifica su FD con `fstat()`.
- **Pratica sicura:** designare le operazioni privilegiati in servizi / demoni ben controllati piuttosto che affidare a utilità `setuid` poco verificate.

24 Smashing the stack, stack frames e controllo del *instruction pointer*

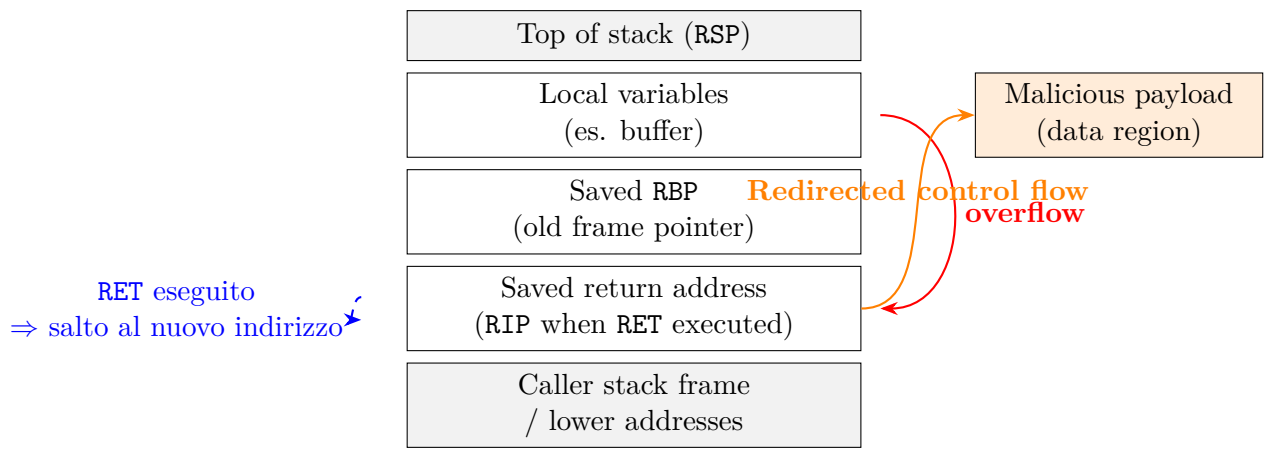
Obiettivo (concetto) Il «*holy grail*» dell'exploitation è ottenere il controllo dell'*instruction pointer* (IP) di un processo sfruttando un bug di sicurezza: in termini generali, si causa la scrittura di un valore controllato dall'attaccante in una locazione che verrà in seguito caricata nel registro di controllo del flusso (es. RIP su x86_64). Il meccanismo tipico è a due fasi: 1) sovrascrittura di un dato sensibile (es. return address salvato sullo stack); 2) esecuzione di un'istruzione legittima (es. RET) che trasferisce il controllo all'indirizzo fornito dall'attaccante.

24.0.1 Concetti di base sullo *stack* e sullo *stack frame*

- **Stack (LIFO):** area di memoria a crescita verso gli indirizzi bassi, gestita con operazioni *push/pop*. Il registro RSP (stack pointer) punta alla cima dello stack.
- **Stack frame:** per ogni chiamata a funzione viene allocato un frame che contiene, tipicamente:
 1. *Saved return address* (indirizzo di ritorno);
 2. *Saved frame pointer* (es. RBP);
 3. registri salvati (callee-saved);
 4. variabili locali (incl. buffer);
 5. eventualmente parametri per chiamate successive.
- **CALL / RET:** CALL mette l'indirizzo di ritorno sullo stack e salta al callee; RET preleva l'indirizzo e torna all'istruzione successiva al CALL.
- **Passing parameters (x86_64 Linux ABI):** primi 6 argomenti in registri RDI, RSI, RDX, RCX, R8, R9; argomenti addizionali su stack.

24.0.2 Elementi principali:

- Puntatore allo Stack (Top): sp (Stack Pointer)/RSP (Stack Pointer)
- Puntatore di Base del Frame: fp (Frame Pointer)/ RBP (Base Pointer)
- Indirizzo di Ritorno (Codice): ra (Return Address)/ RIP (Instruction Pointer)



Indirizzi di memoria: *higher addresses* (in alto) ⇒ *lower addresses* (in basso)

Figura 1: Schema concettuale del buffer overflow: il contenuto del buffer trabocca sovrascrivendo il *return address*, causando una deviazione del flusso di controllo verso una payload malevola. (Diagramma puramente illustrativo.)

24.0.3 Buffer overflow: effetto e conseguenze (conciso)

- **Come avviene (concetto):** una scrittura oltre i limiti di un buffer locale (es. chiamata a *unsafe* API) può sovrascrivere il contenuto del frame, inclusi saved RBP e return address.
- **Conseguenze possibili:**
 1. crash o *segmentation fault* se l'indirizzo non è valido;
 2. *control flow hijack* se l'attaccante fornisce un indirizzo valido o una gadget chain;

- eventuale esecuzione di codice arbitrario (se eseguibile nello spazio dati) oppure *code reuse* (es. ROP — *Return-Oriented Programming*).

ESEMPIO DI ATTACCO1: si modifica la var `cookie` inserendo piu caratteri in `buf` usando la funzione `get` che non controlla di non sfiorare facendo buffer overflow

```
1      #include <stdio.h>
2      int main() {
3          int cookie;
4          char buf[80];
5          printf("buf: %08x cookie: %08x\n", &buf,
6              &cookie);
7          gets(buf); // prende l'input dell'utente
8                     ma non controlla che sia < di 80 byte
9          if (cookie == 0x41424344)
1             printf("you win!\n");
11     }
```

ATTENZIONE: per ottenere 41424344 (in ASCII: ABCD) devo scrivere 44434241 (in ASCII: DCBA) perche gli interi sono scritti in little endian nelle architetture x86 mentre quando sovrascriviamo andiamo in alto da destra verso sinistra.

ESEMPIO DI ATTACCO2: si sovrascrive RIP in modo da saltare la riga `x = 1`

```
1      void function() {
2          char buffer1[4]; // buffer 1 e' un
3                          // puntatore al primo elemento di buffer
4          int *ret;
5          ret = buffer1 + 16; // si mette dentro ret
6                             // l'indirizzo di buffer +16 (indirizzo
7                             // del primo el di buffer1 + 8 per salire
8                             // su RBP + 8 per ssalire su RIP)
9          (*ret) += 16; //ora che si ha l'indirizzo
10                     // di RIP si modifica facendolo saltare a
11                     // 2 righe in piu' (salti x = 1)
12     }
13     void main() {
14         int x = 0;
15         function();
```

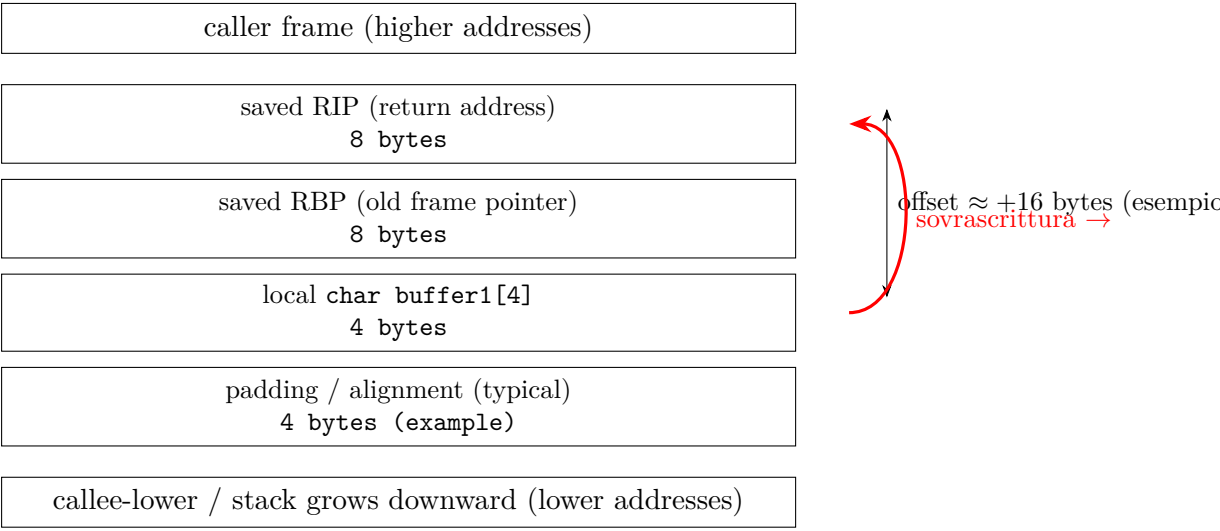
```

10         x = 1;
11         printf("%d\n", x); }

```

ATTENZIONE: Usiamo `char buffer1[...]` negli esempi di overflow perché vogliamo lavorare in unità di byte. Con un array di `char` le operazioni di puntatore si spostano di 1 byte per ogni +1, quindi è facile calcolare un offset preciso sullo stack (es. «16 byte dopo l’inizio del buffer»). Con altri tipi (es. `int`, `long`) l’aritmetica dei puntatori scala per `sizeof(tipo)`, quindi `buffer + 1` non è 1 byte ma 4 o 8 byte — meno comodo per raggiungere byte specifici nello stack.

stack frame x86_64 (addresses: high → low)



Direzione indirizzi: **higher addresses (su) ⇒ lower addresses (giù)**

Nota: i valori mostrati sono *esemplificativi*. L’offset reale dipende da: compiler, ABI, opzioni di compilazione e presenza di variabili locali/salvate.

- In molte ABIs x86_64 il padding per l’allineamento a 16 byte rende l’offset dal buffer al saved RIP tipicamente più grande rispetto a x86_32.
- Per determinare l’offset preciso usare un debugger (es. `gdb`) o stampe degli indirizzi.
- Contromisure comuni: stack canaries, NX (non-executable stack), ASLR, e Control-Flow Integrity (CFI).

Figura 2: Stack frame semplificato per l’esempio: buffer locale, saved RBP e saved RIP.

24.1 Shell vulnerability

: SPOWN SHELL:

questo codice spiega come un programma evochi una shell

```
1      #include <stdio.h>
2      #include <stdlib.h>
3
4      void main() {
5          char *name[2];
6          name[0] = "/bin/sh"; //operazione per
           evocare la shell
7          name[1] = NULL; //termina l'array
8          execve(name[0], name, NULL); //evoca
           execve(path, argv, envp) null per le
           var di ambiente
9          exit(0);
10     }
```

SPOWN SHELL:

in questo caso serve a spostare un programma verso una sezione con del nostro codice per evocare la shell

Shellcode contiene una sfilza di nop (0x90) questo serve perche noi non sappiamo esattamente dove si trova shellcode, aggiungerne tante ci permette di aumentare la probabilità di entrare nella porzione di codice aumentano (le nop saltano fino all'istruzione che ci interessa)

ALTRA NOTA:

il codice della shell code deve essere in binario, questo perche la cpu non compila lo shell code

```
1      Shellcode example
2      char shellcode[] =
           "xeb\x2a\x5e\xc6\x46\x07\x00\x89\x76\x08\xc7\x46\x0c\x00 "
3      "\x00\x00\x00\xb8\x0b\x00\x00\x00\x89\xf3\x8d\x4e\x08\x8d "
4      "\x56\x0c\xcd\x80\xb8\x01\x00\x00\x00\xbb\x00\x00\x00\x00 "
5      "\xcd\x80\xe8\xd1\xff\xff\xff\x2f\x62\x69\x6e\x2f\x73\x68";
6      void main()
7      {
8          int *ret;
9          ret = (int *) &ret + 2;
10         *ret = (int) shellcode; //sovrascrive
           return adres con indirizzo stimato di
           shellcode
```

24.1.1 Misure di mitigazione (panoramica tecnica ma non operativa)

1. **Stack canaries / stack cookies:** valori sentinella inseriti tra variabili locali e saved return address; verificati al *function epilogue*. Se corrotti, il programma abortisce invece di eseguire RET.
2. **Non-executable stack (NX / DEP):** pagine dati marcate non eseguibili; impediscono che codice iniettato nella memoria dati venga eseguito direttamente.
3. **Address Space Layout Randomization (ASLR):** randomizza gli indirizzi di librerie, heap, stack per ridurre l'affidabilità degli indirizzi predittivi; rende più difficile puntare con precisione a indirizzi stabili.
4. **Control-Flow Integrity (CFI):** tecniche che limitano i target ammessi per istruzioni di salto/ritorno a quelli previsti dal *CFG* legittimo (non a runtime si fa uno schema dei salti alle funzioni e si compara con il salto effettivo a runtime).
5. **Forti pratiche di coding:** usare API sicure (es. funzioni bounded), evitare *unsafe* string copy/format, usare analisi statiche/dinamiche, fuzzing e code review.
6. **Least privilege & sandboxing:** eseguire processi con privilegi minimi e confinare servizi in contesti isolati (container, sandbox) per ridurre l'impatto in caso di compromissione.
7. **Compiler mitigations:** attivare opzioni come `-fstack-protector`, link-time protections, e l'uso di librerie sicure.
8. **Runtime monitoring e ASL (application security logging):** monitorare comportamenti anomali, eseguire audit e logging per rilevare tentativi di exploitation.

24.1.2 Evoluzioni dell'exploitation (breve)

Gli attacchi moderni spesso evitano l'iniezione di codice eseguibile (reso difficile da NX/DEP) e preferiscono tecniche di *code reuse* come ROP, che concatenano frammenti di codice già presenti in memoria. Le mitigazioni moderne cercano di combinare più contromisure (canary + NX + ASLR + CFI) per aumentare la difficoltà di successo.

25 Encryption and Decryption (Concetti generali)

25.1 Definizione di cifratura

Encryption (cifratura) e **decryption** (decifratura) sono algoritmi che garantiscono la *data confidentiality*. Sono parametrizzati da una **chiave crittografica** k , rappresentata da una stringa binaria che codifica un numero segreto.

25.2 Plaintext e Ciphertext

- **Plaintext** (m): il messaggio in chiaro.
- **Ciphertext** (c): il messaggio cifrato.

Encryption trasforma m in c , e **Decryption** inverte il processo usando la chiave di decifratura k' .

$$c = E_k(m) \quad m = D_{k'}(c)$$

25.3 Principi

- Solo la chiave rimane segreta (principio *Open Design*).
- I dati sensibili devono essere cifrati prima della trasmissione o memorizzazione.

25.4 Modello generale

- Set di plaintext: P
- Set di ciphertext: C
- Set di chiavi: K
- Funzioni:

$$E : (P \times K) \rightarrow C, \quad D : (C \times K) \rightarrow P$$

25.5 Exhaustive Key Search

Ricerca esaustiva = provare tutte le chiavi $k \in K$:

$$\text{Numero di chiavi} = 2^n$$

Per una chiave di 128 bit, servono in media 2^{127} tentativi -> praticamente impossibile.

25.6 Esempio: DES

- Lunghezza chiave: 56 bit $\rightarrow 2^{56}$ chiavi.
- Oggi non più sicuro (chiavi troppo corte per standard moderni).

25.7 Modelli di attacco

- **Ciphertext-only attack**: l'attaccante ha solo c .
- **Known-plaintext attack**: conosce coppie (m, c) .
- **Chosen-plaintext attack**: sceglie m e osserva c .
- **Chosen-ciphertext attack**: sceglie c e riceve m .

Obiettivo: dedurre la chiave o decifrare nuovi ciphertext.

25.8 Tipi di avversari

- **Passive**: osserva (non altera i dati, prova a decriptare dai dati che osserva).
- **Active**: modifica, inietta o crea comunicazioni per fingersi qualcuno.

26 Symmetric-Key Encryption and Decryption

26.1 Definizione e Logica Simmetrica

La **Symmetric-key encryption** (*secret-key*) si basa sull'utilizzo di un'unica chiave segreta condivisa tra mittente e destinatario:

$$k = k'$$

Il vantaggio principale è la velocità computazionale, ma lo svantaggio risiede nella sfida della gestione e distribuzione sicura della chiave. La cifratura simmetrica si divide principalmente in due filosofie di trattamento dei dati: **a bit** (o a flusso) e **a blocchi**.

26.2 Cifratura a Bit (Bit Cipher) e Stream Ciphers

Questi algoritmi lavorano con un "ritmo" continuo, processando una minima unità di dati alla volta.

- **Meccanismo XOR:** Si basa sull'operazione logica XOR ($\oplus$). La cifratura è $c_i = m_i \oplus k_i$ e la decifratura è $m_i = c_i \oplus k_i$. Lo XOR è ideale perché è un'operazione reversibile e velocissima.
- **One-Time Pad (OTP):** È il caso limite del bit cipher. Se la chiave k è lunga quanto il messaggio, casuale e mai riutilizzata, il sistema è *information-theoretically secure* (incrackabile anche con potenza infinita).
- **Stream Ciphers:** Sono la variante pratica. Poiché scambiarsi chiavi lunghe Gigabyte è impossibile, si usa una chiave corta (*seed*) per generare un **keystream** infinito di bit pseudo-casuali. È come un "rubinetto" di bit che cifrano i dati in tempo reale (es. streaming audio o input tastiera).

26.3 Cifratura a Blocchi (Block Ciphers)

A differenza degli stream ciphers, i block ciphers non lavorano bit per bit ma "impacchettano" i dati in vagoni di dimensione fissa.

- **Funzionamento:** Il messaggio viene diviso in blocchi di lunghezza definita (**Blocklength**, es. 128 bit). La stessa chiave viene applicata sequenzialmente a ogni blocco.
- **Determinismo:** Se applicata a due blocchi identici, la chiave produrrà lo stesso risultato. Questo può rivelare pattern nei dati se non vengono usate modalità di concatenazione (come CBC).
- **Padding:** Se l'ultimo blocco del messaggio non raggiunge la *blocklength*, si aggiungono bit di riempimento. Una tecnica comune è inserire un bit "1" seguito da una sequenza di "0".

26.4 AES (Advanced Encryption Standard)

AES è il moderno standard di riferimento per i block ciphers. Sostituisce il vecchio DES ed è basato sull'algoritmo **Rijndael**.

- Definisce una permutazione matematica complessa per ogni chiave.
- Supporta chiavi di tre lunghezze: 128, 192 e 256 bit. Maggiore è la lunghezza, maggiore è la resistenza ai tentativi di *brute force* (sicurezza computazionale).

26.5 Integrità e MAC (Message Authentication Code)

La cifratura simmetrica garantisce che nessuno legga il messaggio (*Confidenzialità*), ma non impedisce a un attaccante di modificarne i bit durante la trasmissione.

- **MAC:** È un "tag" di autenticazione (simile a un sigillo di garanzia).
- **Processo:** Si passa il testo cifrato e la chiave a una funzione di hash. Il risultato viene appeso al messaggio.
- **Verifica:** Il destinatario ricalcola il tag con la sua chiave: se coincide con quello ricevuto, il messaggio è integro. Questo trasforma la cifratura da *length-preserving* a un formato espanso (messaggio + tag).

Conclusioni

La sicurezza non risiede nel mantenere segreto l'algoritmo (Principio di Kerckhoffs), ma nel proteggere la **chiave**. Gli algoritmi simmetrici moderni come AES, se uniti a controlli di integrità (MAC) e gestione corretta dei blocchi, offrono una protezione robusta e performante per la maggior parte delle infrastrutture digitali.

27 Crittografia Asimmetrica

La **crittografia asimmetrica** utilizza una coppia di chiavi:

- **Chiave pubblica** (K_{pub}): può essere condivisa liberamente.
- **Chiave privata** (K_{priv}): deve rimanere segreta.

Ogni chiave può **crittografare** dati che solo l'altra può **decrittografare**.

27.1 Funzionamento

1. Cifratura per confidenzialità:

- Il mittente cifra il messaggio M con la chiave pubblica del destinatario:

$$C = E_{K_{pub, Dest}}(M)$$

- Solo il destinatario, con la propria chiave privata, può decifrare:

$$M = D_{K_{priv, Dest}}(C)$$

o viceversa

2. Firma digitale per autenticazione: (generalmente si fa l'hash del documento per renderlo più breve)

- Il mittente genera una firma del messaggio con la propria chiave privata:

$$S = \text{Sign}_{K_{priv, Mit}}(\text{hash}(M))$$

- Chiunque può verificare la firma con la chiave pubblica del mittente:

$$\text{Verify}_{K_{pub, Mit}}(S, \text{hash}(M)) \Rightarrow \text{valida o non valida}$$

27.2 Proprietà principali

- Garantisce **confidenzialità**: solo il destinatario può leggere il messaggio.
- Garantisce **autenticità e integrità**: solo il mittente può generare una firma valida. implica la **non repudiabilità** poichè garantisce che solo io ho firmato e non qualcun altro.
- Tipici algoritmi: RSA, DSA, ECC.

28 2.5 Funzioni di hash crittografiche (Cryptographic hash functions)

28.1 Definizione e scopo

Una **cryptographic hash function** H è una funzione efficiente che prende in input una stringa arbitraria m e restituisce un valore a lunghezza fissa

$h = H(m)$ (hash, digest, fingerprint). Serve come *rappresentazione compatta* e *integrità* dei dati.

attenzione: a diff delle funzioni a chiave che sono biunivoche, queste sono suriettive, quindi teoricamente esistono le collisioni ma è computazionalmente impossibile trovarle in un tempo realisticamente utile.

28.2 Proprietà fondamentali

Siano H l'hash e h il valore risultante. Le proprietà di sicurezza desiderate:

- (H1) **One-way / Preimage resistance** Dato h , è computazionalmente infeasible trovare m tale che $H(m) = h$ (non puoi risalire al messaggio originale partendo solo dal valore hash).
- (H2) **Second-preimage resistance** Dato m_1 , è infeasible trovare $m_2 \neq m_1$ con $H(m_2) = H(m_1)$ (se conosci un messaggio, non puoi trovarne un altro diverso che produca lo stesso hash).
- (H3) **Collision resistance** È infeasible trovare *qualsiasi* coppia $m_1 \neq m_2$ con $H(m_1) = H(m_2)$ (due messaggi non otterranno mai lo stesso risultato (sottile rispetto alla precedente dato che H2 è più complesso, magari una coppia è uguale ma non questa m)).

Nota: $H3 \Rightarrow H2$ in pratica; H3 è la proprietà più forte richiesta per firme/dati pubblici. (in **H2** m_1 è fisso, quindi bisogna lavorare attorno per trovarne uno che produca lo stesso hash, in **H3** invece possiamo scegliere noi sia m_1 sia m_2)

28.3 Proprietà pratiche

- Input arbitrario $\rightarrow$ output a lunghezza fissa.
- Ogni piccolo cambiamento in input cambia l'hash in modo imprevedibile (avalanche effect).
- Calcolo efficiente di $H(m)$.
- Non sono richiesti parametri segreti: l'algoritmo è pubblico, la sicurezza dipende dalla resistenza alle proprietà sopra.

28.4 Esempi e famiglia comuni

- **SHA-3**
- **SHA-2**
- **SHA-1**: 160-bit (deprecato per collisioni pratiche).
- **MD5**: 128-bit (deprecato).

28.5 Applicazioni principali

- **Integrity checking** (es. verificare che un eseguibile non sia stato modificato).
- **Password hashing** (non basta un hash semplice: servono salt e funzioni lente o iterazioni).
- **Digital signatures**: si firma l'hash del messaggio (efficienza + input di lunghezza fissa).
- **Merkle trees**: struttura per verificare l'integrità di sottoinsiemi di dati.

28.6 Birthday paradox (motivazione collision-resistance)

La probabilità di trovare una collisione aumenta approssimativamente con $\sqrt{N}$: per un dominio di uscita di 2^n valori, un attacco di tipo *birthday* richiede circa $2^{n/2}$ tentativi. Pertanto una funzione con output di 256 bit fornisce un margine molto più alto rispetto a 128 bit.

28.7 Uso corretto con firme digitali

Quando si firma un messaggio m , si calcola $h = H(m)$ e si firma h con la chiave privata. Per sicurezza, è necessario usare una funzione collision-resistant (H3), altrimenti un attaccante potrebbe far firmare m_1 e sostituirlo con m_2 che ha lo stesso hash.

28.8 Esempio sintetico: verifica integrità file

1. Calcolare $h_0 = H(\text{file})$ al momento di installazione (trusted value).
2. Conservare h_0 in area protetta.
3. Al run-time ricomputare $h = H(\text{file})$ e verificare $h \stackrel{?}{=} h_0$.

28.9 Note pratiche su password hashing

Non usare semplici hash (es. SHA-256) direttamente per password: bisogna applicare:

- **Salt** univoco per ogni password (evita rainbow tables).
- Funzioni **work-intensive** / **memory-hard** (es. Argon2, bcrypt, scrypt) per rallentare attacchi offline.

29 2.6 Message Authentication (Data Origin Authentication)

29.1 Obiettivo

Fornire **integrità** dei dati e **data origin authentication**: il destinatario verifica che il messaggio non sia stato alterato e che provenga da una parte che possiede una chiave condivisa.

29.2 Messaggi e tag (MAC)

Un **MAC** (Message Authentication Code) è una funzione $M_k(m)$ che usa una **chiave segreta** k (simmetrica) per produrre un tag t .

$$t = M_k(m)$$

Il mittente invia la coppia (m, t) . Il ricevente che ha k , ricalcola $M_k(m)$ e verifica t .

29.3 Proprietà desiderate per un MAC

- Impossibilità, per un attaccante privo di k , di forgiare un tag valido per un nuovo messaggio.
- Resistenza a attacchi sia su messaggi scelti che su messaggi osservati.

29.4 Costruzioni comuni di MAC

- **HMAC-H**: costruzione che trasforma un hash (H) in un MAC sicuro (es. HMAC-SHA256). HMAC usa due chiavi derivate e padding interni, evita molte debolezze di concatenazione semplice.

- **CMAC / CBC-MAC**: costruzioni basate su block cipher (AES). CMAC è raccomandato rispetto al semplice CBC-MAC per vari casi di padding e lunghezze variabili.
- **Poly1305** (spesso combinato con AES o ChaCha20).

29.5 MAC vs. Digital Signatures

- **MAC**: simmetrico, fornisce integrità e data-origin tra le parti che condividono la chiave, ma non fornisce **non-repudiation** (chiunque con la chiave può generare il tag).
- **Signature**: costruzione a chiave pubblica che fornisce anche non-repudiation.

29.6 Freshness e replay protection

Un MAC non garantisce di per sé che il messaggio sia “fresco”. Per evitare replay si usano:

- **Nonce** o valori randomici unici;
- **Timestamp** con tolleranza e controllo;
- **Sequence numbers** e controllo stato.

29.7 Combining MAC e Encryption

Due approcci comuni per fornire sia confidenzialità che integrità:

1. **Encrypt-then-MAC**: cifrare il messaggio, quindi calcolare MAC sul ciphertext (raccomandato).
2. **MAC-then-encrypt** (MAC su m in chiaro e poi encrypt su m e MAC) o **Encrypt-and-MAC** (MAC su m in chiaro spedito con m encriptato): hanno rischi e richiedono attenzione al protocol design.

30 Certificati

30.1 Definizione

Un **public-key certificate** è una struttura dati che contiene:

- il nome del soggetto (Subject Name);

- la **public key** appartenente al soggetto;
- una **digital signature** di una **Certification Authority (CA)** che attesta la validità dell'associazione tra nome e chiave pubblica.

30.2 Funzione

La firma della CA fornisce una garanzia che la chiave pubblica appartenga realmente al soggetto indicato. Le parti che fanno affidamento sul certificato (**relying parties**) devono possedere una copia autentica della **public key** della CA per verificarne la firma.

31 Certification Authorities (CA)

31.1 Ruolo

La CA svolge un ruolo critico nel garantire l'affidabilità dei certificati:

- Effettua una **due diligence** (insieme di controlli) per verificare l'identità del soggetto;
- Verifica l'associazione tra il soggetto e la chiave pubblica;
- Può inviare un messaggio di sfida (*challenge*) per confermare il possesso della chiave privata.

31.2 Benefici

La fiducia in una singola CA consente di estendere la fiducia a molti altri certificati firmati da essa.

32 Lunghezze di Chiave Raccomandate (NIST)

32.1 Sicurezza Simmetrica

Il NIST raccomanda almeno **112 bit di security strength** per usi pubblici governativi.

32.2 Equivalenze

n di bit necessari alla chiave da ogni algoritmo per avere lo stesso risultato

Security Strength (bits)	Equivalenti	RSA Modulus	DH Modulus	ECC Key
112	3DES	2048	2048	224–255
128	AES	3072	3072	256–383

Tabella 1: Lunghezze di chiave comparabili tra algoritmi.

33 Elliptic Curve Cryptography (ECC)

33.1 Concetto

Elliptic Curve Cryptography (ECC) implementa funzioni di cifratura, firma digitale e scambio di chiavi utilizzando operazioni su punti di una curva ellittica.

33.2 Vantaggi

- Stessa sicurezza con chiavi più piccole;
- Maggiore efficienza computazionale e di memoria.

33.3 Svantaggi

- Operazioni matematiche più complesse;
- Verifica delle firme (public key operation) più costosa rispetto a RSA.

34 Public-Key Infrastructure (PKI)

34.1 Definizione

Una **PKI** è un insieme di tecnologie e processi per la gestione delle chiavi pubbliche e dei certificati:

1. Strutture dati (certificati, chiavi formattate);
2. Tool crittografici per l'automazione;
3. Componenti architetturali (CAs, directory di certificati);
4. Procedure e protocolli per emissione e aggiornamento dei certificati.

34.2 Scopi

La PKI consente:

- autenticazione delle entità;
- creazione di session keys autenticati;
- firme digitali legalmente riconosciute (non-repudiation).

35 Validazione della Catena di Certificati

35.1 Passaggi di Verifica

Per considerare valido un certificato:

1. la data corrente deve essere nel periodo di validità;
2. il certificato non deve essere revocato;
3. la firma della CA deve essere matematicamente verificabile;
4. la CA deve essere affidabile o parte di una catena valida;
5. il nome nel certificato deve corrispondere all'uso previsto (es. dominio TLS);
6. devono essere rispettati i vincoli nelle estensioni (es. Key Usage, Path Length);
7. se non firmato da un trust anchor, deve esistere una catena valida fino a uno.

35.2 Trust Anchors e Certificati Self-Signed

Trust anchors sono certificati auto-firmati (self-signed) riconosciuti come fidati a priori (es. root CAs nei browser).

36 Estensioni X.509v3

- **Basic-Constraints:** indica se la chiave è CA e limita la lunghezza della catena.

- **Key-Usage / Extended-Key-Usage:** specifica scopi d'uso (firma, cifratura, autenticazione TLS, code signing).
- **Subject-Alternate-Name (SAN):** alternative name forms (email, dominio, IP).
- **Name-Constraints:** limita o permette specifici namespace per i soggetti successivi nella catena.

37 Cross-Certificates

Cross-certificate pairs permettono a due CAs di emettere certificati reciproci, abilitando la fiducia tra diverse gerarchie PKI.

38 Concetti di Fiducia

- La fiducia non è intrinsecamente transitiva.
- Ogni passaggio nella catena deve essere verificato individualmente.

39 Certificate Revocation

39.1 Concetto generale

- I certificati hanno una data di scadenza predefinita (tipicamente 1-2 anni).
- Possono essere **revocati** prima della scadenza (come una carta di credito disattivata).
- La **CA (Certification Authority)** che ha emesso il certificato è responsabile della gestione della revoca.

39.2 Motivazioni della revoca

- Compromissione (o sospetta compromissione) della **chiave privata** del soggetto.
- Sostituzione della chiave con una nuova prima della scadenza.
- Cessazione d'uso della chiave.
- Cambiamento di ruolo o affiliazione del titolare.

39.3 Metodi principali di revoca

39.3.1 Metodo I — Certificate Revocation Lists (CRL)

- La CA pubblica periodicamente una lista firmata e datata di certificati revocati ma non ancora scaduti.
- Modelli:
 - **Push model**: la lista è inviata ai membri.
 - **Pull model**: la lista è resa disponibile per il download.
- Problema: la lista può diventare molto lunga; accorciare la validità dei certificati riduce la dimensione della CRL.

39.3.2 Metodo II — CRL Fragments (Partitions e Deltas)

- Migliora l'efficienza suddividendo la CRL:
 - **Partitioned CRLs**: CRL divisa in sottoliste per range di numeri seriali.
 - **Delta CRLs**: pubblicano solo gli aggiornamenti rispetto a una CRL base.
- Le **CRL Distribution Points** indicano dove trovare le CRL (via HTTP, LDAP, ecc.).

39.3.3 Metodo III — Online Status Checking (OCSP)

- Controllo in tempo reale dello stato del certificato tramite **Online Certificate Status Protocol (OCSP)**.
- **Pull model**: il client interroga un server fidato.
- Variante **OCSP-stapling**: il server stesso fornisce prove aggiornate della validità del proprio certificato.
- Vantaggio: risposta in tempo reale.

39.3.4 Metodo IV — Short-Lived Certificates

- Certificati con durata molto breve (es. 1–4 giorni).
- Evitano completamente la necessità di revoca.
- Svantaggio: aumento dell'overhead di rilascio frequente.

39.3.5 Metodo V — Trusted Public Keys Directly

- Eliminazione completa di certificati firmati.
- I client ottengono chiavi pubbliche valide direttamente da un **trusted key server**.
- Adatto a sistemi chiusi con un'unica autorità amministrativa.

39.4 CA vs End-Entity Revocation

- Sia i certificati CA che quelli degli end-entity possono essere revocati.
- Le **Certification Authority Revocation Lists (CARL)** gestiscono revoche di CA.
- La revoca dei **trust anchors** (radici) avviene tramite aggiornamenti del software (browser, OS).

39.5 Denial of Service (DoS) e Revoca

se il servizio non è raggiungibile (o momentaneamente o a causa di un attacco, esistono 2 vie per l'autorizzazione)

- Se il servizio di revoca è bloccato:
 - **Fail closed**: considera il certificato come revocato (predilige la sicurezza).
 - **Fail open**: considera il certificato valido → vulnerabile (viola il principio **P2 SAFE-DEFAULTS** predilige la disponibilità).

40 CA/PKI Architectures e Certificate Trust Models

40.1 Model IV — Browser Trust Model

i browser hanno la lista dei CA che fungono da radici, ogni CA può autorizzare altre CA intermedie creando una foresta

- Basato su una **foresta di alberi gerarchici disgiunti**.
- Nessun **CA cross-certificate** (ponti e non rami, non bisogna risalire fino a una radice).

- I browser contengono una lista di **trust anchors** (root CA pubbliche).
- Ogni foglia dell'albero corrisponde a un **server TLS**.

40.2 Model V — Enterprise PKI Model (Decentralized CA Trust), siti gov o aziende

gestiscono l'auth internamente

- Le CAs di livello più basso (quelle che firmano i certificati degli utenti) gestiscono la fiducia.
- Possibilità di **cross-certification** tra dipartimenti o aziende.
- Permette **trust peering** più granulare.
- Utilizzato per costruire **comunità di fiducia** in aziende o enti governativi.

40.3 Model VI — User-Control Trust Model (Web of Trust)

l'idea è di creare una rete dove chiedo a chi mi fido di dirmi se una chiave di qualcun altro è corretta.

- Nessuna CA centrale.
- Ogni utente agisce come propria CA (firma i propri certificati).
- La fiducia si propaga tramite connessioni dirette tra utenti (**ad-hoc trust graph**).
- Esempio: **PGP (Pretty Good Privacy)** per email sicure.

40.4 Aspetti chiave

1. Quali trust anchors sono configurati negli end-entity.
2. Quali relazioni di fiducia esistono tra le CAs (chi certifica chi).

41 TLS Website Certificates e Browser Trust Model

41.1 Transport Layer Security (TLS)

- Standard de facto per la sicurezza browser-server.
- Obiettivi principali:
 - Cifratura del traffico (confidenzialità).
 - Autenticazione del server tramite certificati.

41.2 Gradi di certificati TLS

DV (Domain Validated): verifica minima del dominio; emissione automatizzata e a basso costo.

OV (Organization Validated): verifica del dominio + controlli manuali su organizzazione.

EV (Extended Validation): verifica approfondita dell'esistenza legale, fisica e amministrativa dell'organizzazione.

IV (Individual Validated): certificati accettati manualmente dall'utente (es. self-signed).